



Effektiv arbeiten mit Paperclip

Ein praktisches Tutorial für produktive Multi-Agent-Workflows

Autor: Ulrich Cech

Zielgruppe: Entwickler, Architekten, Produktverantwortliche und Solo-Founder, die Paperclip strukturiert einsetzen möchten

Voraussetzung: Eine lauffähige Paperclip-Installation

Quellen (Markdown + Templates): <https://github.com/ulrichcech/linkedin-articles/tree/main/paperclip-workflow>

Erfahrungsbasis: Zwei Wochen aktive Arbeit mit Paperclip im fiktiven Projekt ChronoX. Die Beispiele und Konventionen sind kein „Best Practice nach Jahren Erfahrung“, sondern eine erste, ehrliche Bestandsaufnahme — gedacht als Startpunkt, nicht als Endpunkt.

Stand: 07.05.2026

Worum es in diesem Tutorial geht

Ich habe in den letzten 2 Wochen viel mit Paperclip gearbeitet. Mein erster Reflex war, möglichst viele Agenten anzulegen und ihnen Aufgaben zuzurufen. Das Ergebnis war meistens chaotisch: Die Agenten erfanden Anforderungen, trafen unsichtbare Entscheidungen, bauten am Ziel vorbei.

Was wirklich geholfen hat, war nicht *mehr* KI — und entgegen meiner ersten Vermutung auch kein großzügigeres Token-Limit — sondern eine klarere Arbeitsorganisation. Vision, Rollen, Modi, Handoffs, Reviews — die gleichen Dinge, die ein kleines Team auch braucht.

Dieses Tutorial zeigt anhand eines fiktiven Projekts namens **ChronoX**, wie so ein Setup praktisch aussehen kann. ChronoX ist eine Countdown-App für persönliche Ereignisse — einfach genug, um nachvollziehbar zu bleiben, aber konkret genug für reale Produkt-, Design- und Architekturentscheidungen.

Agenten liefern bessere Ergebnisse, wenn sie nicht raten müssen. Klare Aufgaben, sichtbare Entscheidungen, kleine überprüfbare Schritte — das ist im Wesentlichen das ganze Geheimnis.

So liest Du dieses Tutorial

Die Kapitel 1-4 erklären die Grundidee und die wichtigsten Projektdateien. Kapitel 5-8 zeigen, wie `VISION.md` und `AGENTS.md` aussehen können. Kapitel 9-15 beschreiben Kommunikation und Task-Modes — den Kern der Arbeitsweise. Kapitel 16-20 sind praktische Workflows, Best Practices, Anti-Patterns und eine Prompt-Bibliothek. Kapitel 21-25 enthalten Checklisten, Fazit und Anhänge.

Im laufenden Projekt brauchst Du nicht das ganze Tutorial. Wichtig sind dann vor allem `VISION.md`, `AGENTS.md`, `WORKFLOW.md` und `TASKS.md`.

Inhaltsverzeichnis

1. [Einleitung und Ziel des Tutorials](#)
 2. [Voraussetzungen](#)
 3. [Paperclip wie ein kleines Unternehmen denken](#)
 4. [Die wichtigsten Projektdateien](#)
 5. [Wie sollte eine `VISION.md` aussehen?](#)
 6. [Welche Agenten braucht man?](#)
 7. [Die wichtigsten Agentenrollen im Detail](#)
 8. [Wie sollte eine `AGENTS.md` aussehen?](#)
 9. [Kommunikation in Paperclip: ohne Chatfenster arbeiten](#)
 10. [Task-Modes: Interaktionsmodus pro Aufgabe festlegen](#)
 11. [Direkte Agentenzuweisung vs. Delegation über CEO/Lead Agent](#)
 12. [Wie kommuniziert der Mensch mit Agenten?](#)
 13. [Wie kommunizieren Agenten untereinander?](#)
 14. [Ping-Pong-Arbeit ohne Chatfenster](#)
 15. [Arbeitsmodi als zentrales Best Practice](#)
 16. [Typischer Workflow: von Idee zu fertigem Feature](#)
 17. [Best Practices für produktives Arbeiten mit Paperclip](#)
 18. [Zusätzliche sinnvolle Dateien](#)
 19. [Anti-Patterns](#)
 20. [Praktische Prompt- und Task-Bibliothek](#)
 21. [Vorschlag für einen späteren LinkedIn-Artikel](#)
 22. [Checklisten für die praktische Arbeit](#)
 23. [Fazit](#)
 24. [Anhänge](#)
 25. [Quellen](#)
-

1. Einleitung und Ziel des Tutorials

Dieses Tutorial ist kein Konzeptpapier, sondern ein Arbeitsdokument. Die Beispiele, Vorlagen und Checklisten sind so geschrieben, dass Du sie direkt in ein eigenes Projekt übernehmen und anpassen kannst.

Der Gedanke dahinter ist einfach: Paperclip ist nicht nur ein weiteres Chat-Interface für KI. Es ist eine Steuerungsebene für mehrere spezialisierte Agenten — und so sollte man es auch behandeln.

Statt allein mit einem Assistenten zu chatten, arbeitest Du mit einem kleinen virtuellen Team: Product Owner, Architect, Engineer, Reviewer, QA, UX/UI, Documentation, dazu optional DevOps, Security oder Research.

Das Ziel ist nicht „mehr KI im Workflow“, sondern weniger Kontextverlust, klarere Verantwortlichkeiten, nachvollziehbare Entscheidungen und weniger zufälliges Prompting.

2. Voraussetzungen

Ich gehe davon aus, dass Paperclip bei Dir bereits läuft. Als durchgehendes Beispiel verwenden wir „ChronoX“ — eine fiktive Countdown-App für persönliche Ereignisse, Meilensteine und Ziele.

Wichtig zum Agent-Backend: Paperclip orchestriert Agenten unterschiedlicher Herkunft („Bring Your Own Agent“ — Claude Code, Cursor, Codex, Bash-Skripte oder HTTP-Webhooks). Für die in Kapitel 16 beschriebenen Repo-Schreibvorgänge — Spezifikationen nach `docs/design/`, ADRs nach `docs/decisions/`, Code-Änderungen — brauchst Du Agent-Typen mit **Schreibzugriff aufs Repo** (Cursor, Claude Code, Codex). Reine Webhook-Agenten können Markdown-Inhalte nur als Work Product zurückliefern, ohne sie selbst zu committen — dann übernimmt der Documentation Agent (oder Du selbst) das Einchecken.

Was dieses Tutorial **nicht** behandelt: Installation, Deployment, API-Key-Konfiguration, Serverbetrieb oder Infrastruktur-Härtung.

Worum es **stattdessen** geht: Wie strukturierst Du die Arbeit mit Paperclip? Welche Agenten brauchst Du? Wie beschreibst Du Vision, Rollen und Kommunikation? Wie arbeitest Du interaktiv, ohne in Chaos und Kostenexplosion zu landen?

3. Paperclip wie ein kleines Unternehmen denken

Paperclip ist kein Chatbot-System. Wenn man es so behandelt, kommt schnell so etwas heraus:

„Ich habe fünf Agenten und jeder darf irgendwie alles machen.“

Mein Setup hat erst funktioniert, als ich die Perspektive gewechselt habe:

„Ich habe eine kleine Organisation mit einer Produktvision, definierten Rollen, Verantwortlichkeiten, Entscheidungswegen und Qualitätskontrollen.“

Praktisch heißt das: Jeder Agent hat eine Aufgabe und Grenzen. Größere Arbeiten haben ein Ziel. Änderungen haben eine Qualitätskontrolle. Du selbst bleibst Board, Product Owner und finaler Entscheider.

Eine gute Paperclip-Struktur ist deshalb weniger ein Chatverlauf, sondern eher ein nachvollziehbarer Arbeitsprozess — mit Rollen, Aufgaben, Übergaben und Freigaben.

3.1 Was Paperclip out-of-the-box mitbringt

Damit das Bild greifbar wird, hier die Bausteine, die Paperclip ohne weitere Konfiguration bereitstellt — auf ihnen baut alles Folgende auf:

- **Tasks** als zentrale Arbeitseinheit mit Kommentaren, Anhängen, Work Products, Labels und expliziten **Blocker-Dependencies** zwischen Tasks.
- **Goals-Hierarchie**: Jeder Task führt auf ein Goal zurück, jedes Goal auf ein Project, jedes Project auf die Company Mission. Das ist nicht Hierarchie um ihrer selbst willen — es zwingt einen, Tasks immer mit einem Ziel zu verknüpfen.
- **Org Chart** mit Rollen, Titeln, Reporting-Lines und Permissions. Delegation läuft offiziell up und down im Org Chart, nicht ad hoc.
- **Heartbeats** als Trigger: Agenten arbeiten nicht synchron, sondern wachen periodisch auf, prüfen ihre Tasks und handeln. Genau deshalb funktioniert Paperclip ohne Chat — der Heartbeat ersetzt das Tippen im Chatfenster.
- **Budgets pro Agent** mit harter Enforcement: erreicht ein Agent sein monatliches Cost-Limit, stoppt er. Ein eigenständiges Steuerungsinstrument, das man im Setup nicht vergessen sollte.
- **Audit Log**: jede Instruktion, jede Antwort, jeder Tool-Call und jede Entscheidung wird festgehalten. Die technische Grundlage dafür, dass dieses Tutorial überhaupt von „nachvollziehbarer Arbeit“ reden kann.
- **Bring Your Own Agent**: Paperclip orchestriert, die Agenten selbst können Claude Code, Cursor, Codex, Bash-Skripte oder HTTP-Webhooks sein. Du wählst pro Rolle, was passt.
- **Review- und Approval-Stages** als eingebauter Governance-Schritt vor einer Freigabe.

Worauf das Tutorial dagegen *nicht* zurückgreift, weil es **Konventionen** sind und keine Paperclip-Primitive: konkrete Status-Werte (siehe `WORKFLOW.md`), die `MODE: PROPOSE/ITERATIVE/EXECUTE`-Schreibweise, die Markdown-Templates für Handoff, Review Request und Approval Request sowie die Aufteilung in `VISION.md` / `WORKFLOW.md` / `DECISIONS.md`. Diese Konventionen setzen alle auf Paperclips Tasks, Kommentaren, Goals und Approval-Stages auf — sie sind keine Konkurrenz, sondern eine Schreibweise auf der vorhandenen Mechanik.

4. Die wichtigsten Projektdateien

Eine kleine, klar abgegrenzte Menge Projektdateien hat sich für mich als ausreichend erwiesen.

Mindestens sinnvoll:

```
/VISION.md
/AGENTS.md
/WORKFLOW.md
/TASKS.md
/HEARTBEAT.md
```

Sobald das Projekt ernsthafter wird, kommen meist noch dazu:

```
/ARCHITECTURE.md
/DESIGN.md
/DECISIONS.md
/SECURITY.md
/RELEASE.md
/SKILLS.md
```

Mein Rat: nicht alles auf einmal anlegen. Starte mit den ersten fünf und ergänze Architektur, Design und Entscheidungen erst dann, wenn Du sie wirklich vermisst.

4.1 VISION.md

Beschreibt, **warum** das Projekt existiert und **was** entstehen soll. Nicht primär technisch, sondern die strategische Leitplanke für alle Agenten.

4.2 AGENTS.md

Beschreibt, **wie** Agenten in diesem Projekt arbeiten sollen: Projektkontext, Rollen, Verantwortlichkeiten, Arbeits- und Kommunikationsregeln, Build- und Testbefehle, Sicherheits- und Reviewregeln.

Verhältnis zum offenen `agents.md`-Format: Das unter [agents.md](#) standardisierte Format definiert `AGENTS.md` enger — primär als „Instructions for AI coding agents“ mit Setup-Befehlen, Coding-Style und Build-/Test-Anleitungen. Paperclip orientiert sich daran, die hier vorgeschlagene Struktur ist breiter (zusätzlich Rollen, Modi, Eskalation). Das ist kein Widerspruch — der enge `agents.md`-Kern (Build-/Test-/Setup-Befehle) bleibt ein sinnvoller Bestandteil und sollte in der eigenen `AGENTS.md` enthalten sein, ergänzt um die hier beschriebenen Themen.

4.3 WORKFLOW.md

Beschreibt den konkreten Arbeitsprozess: Wann darf ein Agent direkt Code ändern? Wann muss er erst Vorschläge liefern? Wann ist ein Review erforderlich? Wann musst Du selbst entscheiden? Wie kommunizieren Agenten miteinander, und wie werden Blocker, Rückfragen und Handoffs dokumentiert?

4.4 TASKS.md

Wichtige Abgrenzung: `TASKS.md` ist *kein* Mirror der Paperclip-Task-Liste. Paperclip selbst ist die Quelle der Wahrheit für Status, Assignee und Blocker-Dependencies — diese parallel in einer Markdown-Datei zu pflegen, würde nur Doppelarbeit erzeugen, und beide Quellen würden auseinanderlaufen.

`TASKS.md` enthält stattdessen nur das, was Paperclip nicht liefert: ein menschenlesbares **Narrativ** — was läuft gerade, welche Schritte stehen als Nächstes an, welche Themen wurden bewusst zurückgestellt, welche Entscheidungen sind in der jüngeren Vergangenheit gefallen. Wer sehen will, an welchem Task der Engineer gerade arbeitet, schaut in Paperclip. Wer den Projektkontext in zwei Minuten erfassen will, liest `TASKS.md`. Ein vollständiges Beispiel im Anhang.

4.5 HEARTBEAT .md

Heartbeats sind das Paperclip-Primitiv, das Agenten überhaupt erst zum Arbeiten bringt: ohne Heartbeat kein Trigger, keine ausgeführten Tasks. Damit das nicht im Verborgenen passiert und sich nicht zur Kostenfalle entwickelt, lohnt es sich, die Heartbeat-Konfiguration **explizit in einer eigenen Datei zu dokumentieren** — Taktung pro Agent, was bei jedem Heartbeat passieren soll, Pause-Regeln und Kostenbudget-Bezug. Details und ein Beispiel in Kapitel 18.6.

Hinweis zur Herkunft: `HEARTBEAT.md` ist ursprünglich von **OpenClaw** eingeführt — dort als kurze, kompakte **Heartbeat-Checkliste**, die der Agent bei jedem Heartbeat in seinen Prompt-Kontext lädt (siehe docs.openclaw.ai/gateway/heartbeat; Maxime „klein und stabil halten, um Prompt-Bloat zu vermeiden“). Das hier vorgeschlagene Dokument **erweitert die Idee zu einem Konfigurations- und Koordinationsdokument für mehrere Agenten** — Taktung, Budgets, Pause-Regeln. Beide Konzepte können nebeneinander existieren: eine kurze Agent-Checkliste (OpenClaw-Stil) als Prompt-Beigabe und das ausführlichere Setup-Dokument für menschliche Leser.

4.6 ARCHITECTURE .md

Hält die technischen Grundentscheidungen fest: Architekturtyp, Modulstruktur, Schnittstellen, Datenmodell, Technologien — und ausdrücklich auch die Grenzen und Nicht-Ziele.

4.7 DESIGN .md

Bei UI-Projekten unverzichtbar. Enthält visuelle Richtung, Farben, Typografie, Komponentenlogik, Layoutregeln, gute und schlechte Beispiele, UX-Prinzipien.

4.8 DECISIONS .md

Dokumentiert wichtige Entscheidungen mit Datum, Kontext, Alternativen, Begründung und Konsequenzen. Verhindert vor allem eines: dass Agenten Wochen später eine längst geführte Diskussion neu aufrollen.

4.9 SECURITY .md

Hält die sicherheitsrelevanten Spielregeln für das Projekt fest: was Agenten niemals tun dürfen, wo Secrets liegen, welche Logs welche Daten enthalten dürfen, welche Aktionen menschliche Freigabe brauchen. Je nach Projekt gehören hier auch das Threat-Modell, der Umgang mit Authentifizierungs-Tokens und Eskalationswege bei verdächtigen Vorfällen rein. Die `AGENTS.md` referenziert diese Datei in ihren Sicherheitsregeln — Doppelpflege vermeidet man, indem die Details hier stehen und `AGENTS.md` darauf verweist.

4.10 RELEASE .md

Beschreibt, wie aus „Feature ist fertig“ ein Release wird: Versionsschema, Changelog-Format, Pre-Release-Checkliste, Deploy-Schritte, wer freigeben darf. Für ChronoX ohne Backend ist das vor allem die Build- und Store-Submission-Pipeline. Bei größeren Projekten kommen Smoke-Tests, Migrationsfreigaben und Roll-Back-Pläne hinzu. Wichtig: `RELEASE.md` hält den Prozess fest, nicht den Inhalt eines konkreten Releases — der gehört in den Changelog oder in einen Release-Task.

4.11 SKILLS .md

Optional, aber von Paperclip offiziell unterstützt: `SKILLS.md` ist eine Datei, die Agenten zur Laufzeit wiederverwendbare Workflows und Skills injiziert — z. B. „Wie führe ich ein Code-Review durch“ oder „Wie schreibe ich einen ADR“. Während `AGENTS.md` das *Setup* beschreibt (Wer ist wer, welche Regeln gelten?), liefert `SKILLS.md` die *Vorgehensweisen*, die mehrere Agenten anwenden sollen. Für kleine Projekte verzichtbar, für ernsthafte Setups eine sinnvolle Ergänzung.

5. Wie sollte eine **VISION.md** aussehen?

Die **VISION.md** darf nicht zu kurz sein, aber sie ist auch kein Roman. Für ein kleines bis mittleres Projekt sind ein bis sechs Seiten ein guter Rahmen — alles über zehn Seiten wird selten gelesen.

Entscheidend ist: Die Vision muss konkret genug sein, dass Agenten daraus Entscheidungen ableiten können.

Schlecht:

```
# Vision

Wir bauen eine tolle Countdown-App.
```

Besser:

```
# Vision

Wir entwickeln ChronoX, eine Countdown-App für persönliche Ereignisse, Meilensteine und Ziele.

ChronoX soll nicht wie ein nüchterner Kalender wirken, sondern wie ein hochwertiger persönlicher Begleiter für Dinge, auf die man sich freut oder hinarbeitet – Geburtstage, Reisen, Prüfungen, Projektstarts.

Die App konkurriert bewusst nicht mit vollwertigen Kalender- oder PM-Tools, sondern bedient Nutzer, die einzelne wichtige Zeitpunkte sichtbar, schön und motivierend verfolgen möchten.
```

Das Vollständige ChronoX-Beispiel — mit Zielgruppe, Nicht-Zielen, Kernfunktionen, Qualitätskriterien und Entscheidungsprinzipien — findest Du im Anhang als **attachments/VISION.md**. Es ist der praktische Gegen-Beweis, dass eine Vision nicht abstrakt bleiben muss.

5.1 Empfohlene Struktur für **VISION.md**

```
# VISION.md

## 1. Produktidee

Was soll entstehen?

## 2. Zielgruppe

Für wen ist das Produkt gedacht?

## 3. Problem

Welches konkrete Problem wird gelöst?

## 4. Nutzenversprechen

Warum sollte jemand dieses Produkt verwenden?

## 5. Nicht-Ziele

Was soll bewusst nicht gebaut werden?

## 6. Kernfunktionen

Welche Funktionen sind essenziell?

## 7. Qualitätskriterien
```


Woran erkennt man, dass das Produkt gut ist?

8. UX-Prinzipien

Wie soll sich das Produkt anfühlen?

9. Technische Leitplanken

Welche Technologien, Plattformen oder Architekturprinzipien sind gesetzt?

10. Business- oder Veröffentlichungsziel

Ist es ein MVP, ein internes Tool, ein SaaS, eine App, ein Demo-Projekt?

11. Entscheidungsprinzipien

Wie sollen Agenten entscheiden, wenn mehrere Optionen möglich sind?

5.2 Beispiel für eine gute **VISION.md**

Das folgende Beispiel ist eine **kompakte Version** für ChronoX. Die ausführlichere Fassung mit detaillierter Nicht-Ziele-Liste, Sekundär-Zielgruppen und ergänzten Kapiteln liegt im Anhang als `attachments/VISION.md` und ist die kanonische Version, falls beide voneinander abweichen.

```
# VISION.md
```

1. Produktidee

Wir entwickeln ChronoX, eine moderne Countdown-App für persönliche Ereignisse, Meilensteine und Ziele.

ChronoX hilft dabei, wichtige Zeitpunkte sichtbar, emotional und motivierend zu verfolgen: Geburtstage, Reisen, Prüfungen, Projektstarts, persönliche Ziele oder besondere Lebensereignisse.

Die App soll nicht wie ein nüchterner Kalender wirken, sondern wie ein hochwertiger persönlicher Begleiter für Dinge, auf die man sich freut oder hinarbeitet.

2. Zielgruppe

Primäre Zielgruppe:

- Menschen, die persönliche Ereignisse bewusst verfolgen möchten
- Nutzer, die Vorfreude und Motivation visuell erleben möchten
- Schüler, Studierende und Berufstätige mit wichtigen Deadlines
- Familien, Paare und Einzelpersonen mit besonderen Meilensteinen

Sekundäre Zielgruppe:

- Menschen, die Ziele und Gewohnheiten emotionaler planen möchten
- Nutzer, die minimalistische, schöne Apps bevorzugen
- Personen, die keine komplexe Projektmanagement-App verwenden möchten

3. Problem

Viele Countdown-Apps wirken technisch, altmodisch oder überladen. Sie zeigen zwar eine verbleibende Zeit an, erzeugen aber wenig emotionale Bindung.

ChronoX schließt diese Lücke: Countdowns nicht nur verwalten, sondern sichtbar, schön und motivierend machen.

4. Nutzenversprechen

Der Nutzer erhält:

- eine schnelle Übersicht über wichtige Ereignisse,
- visuell starke Countdown-Karten,
- eine einfache Möglichkeit, neue Ereignisse anzulegen,
- Kategorien oder visuelle Marker für unterschiedliche Ereignistypen,
- später optionale Widgets, Erinnerungen und Meilenstein-Funktionen.

5. Nicht-Ziele

Die erste Version ist kein vollständiger Kalenderersatz, kein Projektmanagement-Tool und enthält keine Team-, Backend-, Social-Media- oder Gamification-Funktionen.

6. Kernfunktionen für V1

- Countdown-Liste
- Countdown-Karte für ein Ereignis
- Ereignis erstellen, bearbeiten, löschen
- Ziel-Datum und optional Ziel-Uhrzeit
- Kategorie oder visuelle Markierung
- lokale Speicherung
- ansprechendes App-Design für Android und iOS

7. Qualitätskriterien

Das Produkt ist gut, wenn ein Countdown in wenigen Sekunden erstellt werden kann, die App sofort verständlich ist, die Karten visuell hochwertig wirken, wichtige Ereignisse schnell scanbar sind, die App auch ohne Backend zuverlässig läuft und die UI auf Android und iOS konsistent wirkt.

8. UX-Prinzipien

- Emotional, aber nicht verspielt
- Visuell stark, aber nicht überladen
- Schnell erfassbar, wenige Eingaben
- Lesbarkeit vor reiner Ästhetik
- V1 bewusst einfach halten

9. Technische Leitplanken

- App: Kotlin Multiplatform / Compose Multiplatform
- Zielplattformen: Android und iOS
- Datenhaltung: lokal auf dem Gerät
- Backend: keines für V1
- Architektur: modular, testbar, erweiterbar
- Design: hochwertige Cards, klare Typografie, moderne Gradients

10. Veröffentlichungsziel

Eine erste öffentlich testbare App-Version, die Countdowns visuell attraktiv und zuverlässig abbildet. Langfristig erweiterbar um Widgets, Erinnerungen, Meilenstein-Ansichten, Templates, Themes und Export-/Sharing-Funktionen.

11. Entscheidungsprinzipien

Wenn zwischen Einfachheit und Funktionsumfang entschieden werden muss, gewinnt für V1 die Einfachheit.

Wenn zwischen visueller Wirkung und Lesbarkeit entschieden werden muss, gewinnt die Lesbarkeit.

Wenn eine Änderung eine Backend-Abhängigkeit einführt, muss sie für V1 besonders begründet werden.

Wenn ein Feature eher „nice to have“ ist, wird es in eine spätere Version verschoben.

6. Welche Agenten braucht man?

Mein Tipp aus eigener Erfahrung: nicht mit zu vielen Agenten starten. Fünf bis sieben sind ein guter Anfang.

6.1 Minimal sinnvolles Setup

Agent	Zweck
Product Owner Agent	Klärt Anforderungen, formuliert Features, priorisiert
Architect Agent	Achtet auf Architektur, Modulgrenzen, technische Konsistenz
Engineer Agent	Implementiert Features
Review Agent	Prüft Code, Qualität, Risiken und Tests
QA/Test Agent	Erstellt Testfälle, prüft Akzeptanzkriterien
UX/UI Agent	Arbeitet an Design, Flows, Screens und Nutzerführung
Documentation Agent	Dokumentiert Entscheidungen, Nutzung und technische Konzepte

Für sehr kleine Projekte reichen mir oft drei: Product Owner, Engineer, Reviewer. Sobald es ernsthafter wird, kommt der Architect dazu — ab da bin ich bei dem Setup, mit dem ich am verlässlichsten arbeite.

7. Die wichtigsten Agentenrollen im Detail

Für jede Rolle skizziere ich kurz Aufgabe, Grenzen und einen typischen Auftrag. Die Grenzen sind dabei oft wichtiger als die Aufgaben — ohne sie franst der Scope schnell aus.

7.1 Product Owner Agent

Klärt Anforderungen, strukturiert Features, formuliert Akzeptanzkriterien, identifiziert offene Fragen und schlägt Prioritäten vor.

Was er nicht darf: ungefragt technische Architektur festlegen, Code ändern oder offene Produktentscheidungen einfach erfinden.

Typischer Auftrag:

Analysiere diese Feature-Idee.
Formuliere 3 mögliche Umsetzungsrichtungen.
Bewerte sie nach Aufwand, Nutzen und Risiko.
Empfehl eine Richtung, aber triff keine finale Entscheidung.

7.2 Architect Agent

Definiert die technische Struktur, bereitet Architekturentscheidungen vor, erkennt Risiken, schützt Modulgrenzen und macht technische Schulden sichtbar. Schlägt ADRs vor und vergleicht Optionen.

Was er nicht darf: ohne Rückfrage große Architekturwechsel durchziehen, unnötig komplexe Lösungen einführen oder produktstrategische Entscheidungen treffen.

Typischer Auftrag:

Bewerte diese Feature-Anforderung aus Architektursicht.
Zeige 2–3 mögliche technische Lösungswege.
Bewerte sie nach Einfachheit, Erweiterbarkeit, Testbarkeit und Risiko.
Empfehl eine Lösung für V1.
Ändere noch keinen Code.

7.3 Engineer Agent

Implementiert Features, ergänzt Tests, behebt Bugs, macht kleinere Refactorings — und respektiert dabei die bestehende Architektur.

Was er nicht darf: den Produktumfang erweitern, Architektur eigenmächtig ändern, Designentscheidungen erfinden oder Tests entfernen, nur damit der Build wieder grün wird.

Typischer Auftrag:

Implementiere dieses Ticket gemäß Akzeptanzkriterien.
Halte Dich an ARCHITECTURE.md und AGENTS.md.
Führe passende Tests aus.
Dokumentiere am Ende:
1. geänderte Dateien,
2. wichtigste Entscheidungen,
3. ausgeführte Tests,
4. offene Risiken.

7.4 Review Agent

Prüft Änderungen auf Architekturverstöße, bewertet Testabdeckung, identifiziert Sicherheitsrisiken und markiert unnötige Komplexität. Schreibt Review-Kommentare und kann Merge-Freigaben empfehlen oder verweigern.

Was er nicht darf: große Umbauten unaufgefordert selbst vornehmen oder persönliche Vorlieben als harte Regeln verkaufen.

Typischer Auftrag:

Reviewe die Änderungen dieses Tickets.

Prüfe insbesondere:

- Architekturkonformität
- Testabdeckung
- Lesbarkeit
- Security-Aspekte
- unnötige Komplexität
- Abweichungen von AGENTS.md

Gib das Ergebnis als Review mit folgenden Kategorien aus:

- Blocker
- Should Fix
- Nice to Have
- Positive Beobachtungen

7.5 QA/Test Agent

Formuliert Testfälle, findet Edge Cases, prüft Akzeptanzkriterien und erkennt Regressionen.

Was er nicht darf: Anforderungen beliebig erweitern oder Produktverhalten ohne Abstimmung ändern.

Typischer Auftrag:

Leite aus dieser Feature-Beschreibung konkrete Testfälle ab.

Unterscheide:

- Happy Path
- Edge Cases
- Fehlerfälle
- Regressionstests
- manuelle Prüfungen

7.6 UX/UI Agent

Verantwortet Nutzerführung, Screen-Struktur, Designvarianten, Interaktionslogik und visuelle Konsistenz. Schlägt Designoptionen vor, beschreibt Varianten, bereitet Wireframes oder Komponentenstrukturen vor.

Was er nicht darf: ohne Rückfrage ein Designsystem umkrempeln, technische Implementierung erzwingen oder UX-Entscheidungen vorzeitig als erledigt markieren.

Typischer Auftrag:

Erarbeite 3 unterschiedliche UI-Richtungen für dieses Feature.

Beschreibe je Richtung:

- Grundidee
- Layout
- Interaktion
- Vorteile
- Nachteile
- wann diese Richtung geeignet ist

Noch keinen Code ändern.

7.7 Documentation Agent

Dokumentiert Entscheidungen, schreibt Tutorials, aktualisiert das README, pflegt Nutzer- und Entwicklerdokumentation.

Was er nicht darf: Fakten erfinden, Features dokumentieren, die nicht implementiert sind, oder Marketing mit echtem Funktionsumfang vermischen.

8. Wie sollte eine AGENTS.md aussehen?

Die AGENTS.md ist operativer als die VISION.md. Während die Vision sagt, was wir bauen, sagt die AGENTS.md, wie die Agenten in diesem konkreten Projekt arbeiten sollen.

8.1 Empfohlene Struktur

```
# AGENTS.md

## 1. Projektkontext
## 2. Wichtige Dateien
## 3. Grundregeln
## 4. Rollen und Verantwortlichkeiten
## 5. Arbeitsmodi
## 6. Kommunikationsregeln
## 7. Code-Regeln
## 8. Qualitätsregeln
## 9. Sicherheitsregeln
## 10. Eskalationsregeln
## 11. Ausgabeformat
```

8.2 Beispiel für eine AGENTS.md

```
# AGENTS.md

## 1. Projektkontext

Dieses Projekt entwickelt ChronoX, eine moderne Countdown-App für persönliche Ereignisse, Meilensteine und Ziele.

Die Produktvision steht in `VISION.md`.
Die technische Architektur steht in `ARCHITECTURE.md`.
Design- und UX-Regeln stehen in `DESIGN.md`.
Wichtige Entscheidungen werden in `DECISIONS.md` dokumentiert.

Alle Agenten müssen diese Dateien berücksichtigen, bevor sie größere Änderungen vorschlagen oder durchführen.

## 2. Wichtige Dateien

Vor jeder größeren Aufgabe lesen:

- `VISION.md`
- `WORKFLOW.md`
- `SECURITY.md`, sobald sicherheits-, datenschutz- oder dependencyrelevante Themen berührt werden
- `HEARTBEAT.md`, falls Taktungs- oder Budgetfragen relevant sind
- `ARCHITECTURE.md`
- `DESIGN.md`, falls UI betroffen ist
- `DECISIONS.md`, falls Architektur oder Produktverhalten betroffen ist
- `TASKS.md`, falls Aufgabenstatus relevant ist

## 3. Grundregeln

- Keine erfundenen Anforderungen.
- Keine ungefragten Architekturwechsel.
- Keine Entfernung von Tests, nur um Fehler zu beseitigen.
- Keine großen Refactorings innerhalb kleiner Feature-Tickets.
- Änderungen müssen klein, nachvollziehbar und testbar bleiben.
- Bei Unsicherheit zuerst Optionen formulieren, nicht direkt implementieren.
```

4. Rollen

Product Owner Agent

Verantwortlich für Feature-Klärung, Akzeptanzkriterien, Priorisierung, offene Fragen.
Darf keine technischen Implementierungsdetails erzwingen.

Architect Agent

Verantwortlich für Architekturentscheidungen, Modulgrenzen, technische Risiken, ADR-Vorschläge.
Darf keine Produktentscheidungen final treffen.

Engineer Agent

Verantwortlich für Implementierung, Tests, kleine Refactorings, Umsetzung gemäß Architektur.
Darf keine ungeklärten Produktentscheidungen erfinden.

Review Agent

Verantwortlich für Code Review, Architektur-, Test- und Sicherheitsprüfung.
Darf Änderungen blockieren, wenn Blocker gefunden werden.

UX/UI Agent

Verantwortlich für Screen-Entwürfe, Nutzerführung, Designvarianten, Komponentenlogik.
Darf vor finaler Designentscheidung keinen produktiven Code ändern.

5. Arbeitsmodi

Analysemodus

Nur analysieren, keine Codeänderungen. Ausgabe: Beobachtungen, Risiken, offene Fragen, Empfehlung.

Vorschlagsmodus

2-3 Optionen mit Beschreibung, Vor-/Nachteilen, Aufwand, Risiken und Empfehlung. Keine Codeänderungen.

Umsetzungsmodus

Code-Änderungen erlaubt, sofern Ticket klar, Akzeptanzkriterien vorhanden, Architekturregeln klar und keine Produktentscheidungen offen sind.

Reviewmodus

Bestehende Änderungen prüfen, keine großen Umbauten. Ausgabe: Blocker, Should Fix, Nice to Have, Freigabeempfehlung.

6. Kommunikationsregeln

Paperclip ist kein Chat-System. Agenten kommunizieren über Aufgaben, Kommentare, @mentions, Handoffs, Review Requests, Approval Requests, Work Products und Aktualisierungen an Projektdokumenten.

Jede Übergabe enthält Kontext, Ziel, bisherige Entscheidungen, offene Fragen, erwartetes Ergebnis, relevante Dateien und Einschränkungen.

7. Code-Regeln

- Bestehende Architektur respektieren
- Neue Logik muss testbar sein
- Komplexität vermeiden, keine toten Abhängigkeiten, keine globalen Workarounds
- Keine stillen Fehlerbehandlungen
- Keine Secrets in Code oder Logs

8. Tests

Vor Abschluss einer Implementierung relevante Unit- und ggf. Integrationstests ausführen. Bei UI-Änderungen manuelle Prüfschritte dokumentieren. Falls Tests nicht ausführbar sind, Grund dokumentieren.

9. Definition of Done

Eine Aufgabe ist nur fertig, wenn Akzeptanzkriterien erfüllt sind, relevante Tests ergänzt oder angepasst wurden, bestehende Tests nicht absichtlich deaktiviert wurden, geänderte Dateien genannt sind, wichtige Entscheidungen dokumentiert sind, offene Risiken benannt sind und Review durchgeführt oder angefordert wurde.

10. Sicherheitsregeln

Agenten dürfen niemals Secrets ausgeben, API Keys in Dateien schreiben, Authentifizierung umgehen, produktive Daten löschen, Security Checks entfernen oder Logs mit sensiblen Daten erzeugen.

11. Eskalationsregeln

Der Mensch muss gefragt werden, wenn mehrere Produktvarianten möglich sind, UX-Entscheidungen unklar sind, Architekturentscheidungen langfristige Folgen haben, Kosten oder externe Dienste betroffen sind, Datenmodelländerungen nötig sind, Migrationen produktive Daten betreffen oder Sicherheitsrisiken entstehen.

12. Abschlussbericht

Jeder Agent liefert am Ende: Zusammenfassung, geänderte Dateien, getroffene Entscheidungen, ausgeführte Tests, offene Fragen, empfohlene nächste Schritte.

9. Kommunikation in Paperclip: ohne Chatfenster arbeiten

9.1 Die wichtigste Klarstellung

Paperclip ist nicht primär ein Chat-System. Das ist die häufigste Fehlannahme — und gleichzeitig die, die mich am Anfang am meisten Zeit gekostet hat.

Ein zentraler, typischer Fehler ist, Paperclip-Kommentare wie einen Chat zu benutzen:

```
Kommentar: "Hey, mach mal..."
Agent: "Okay, soll ich X oder Y?"
Kommentar: "X klingt gut."
Agent: "Und was ist mit Z?"
Kommentar: "Mach einfach."
```

Das funktioniert eine Weile, fällt aber spätestens beim ersten Handoff auseinander: kein klarer Status, keine entscheidbare Frage, kein nachvollziehbares Ergebnis.

Die Paperclip-Variante sieht stattdessen so aus:

```
Mensch erstellt Aufgabe / Issue / Goal.
Agent wird zugewiesen oder erwähnt.
Agent arbeitet beim nächsten Trigger / Heartbeat.
Agent schreibt Ergebnis, Kommentar, Status oder Work Product.
Mensch oder anderer Agent reagiert über Kommentar, neue Aufgabe, Review oder Approval.
```

Eine „Nachricht“ ist in Paperclip also fast immer ein Task, Kommentar, Handoff, Mention, Review Request, Work Product oder Approval Request — strukturierte Arbeitskommunikation, kein Chat.

9.2 Die konkreten Kommunikationskanäle

In der Praxis lassen sich sechs Formen unterscheiden. Die drei wichtigsten zuerst.

9.2.1 Aufgabe / Issue

Der wichtigste Kanal überhaupt. Du oder ein Agent erstellt eine Aufgabe für einen anderen Agenten.

```
## Task: UI-Varianten für Countdown-Karte entwickeln

### Assigned to
UX/UI Agent

### Kontext
Wir bauen ChronoX, eine moderne Countdown-App für persönliche Ereignisse, Meilensteine und Ziele.

### Ziel
Bitte entwickle 3 unterschiedliche UI-Richtungen für die Event-Countdown-Karte.

### Erwartete Ausgabe
- Variante A: minimalistisch und sehr klar
- Variante B: hochwertig mit Gradient / Glassmorphism
- Variante C: emotional mit Kategorie-Icons oder Illustrationselementen

### Einschränkungen
- Noch keinen Code ändern.
- Keine finalen Designentscheidungen treffen.
- Ergebnis als Kommentar oder Work Product dokumentieren.

### Entscheidung durch Mensch erforderlich
```

Ja.

Im Grunde die Paperclip-Version von „Hey UX, mach mal 2-3 Vorschläge“ — nur eben als strukturierte Aufgabe statt als Chatnachricht.

9.2.2 Kommentar an einer Aufgabe

Hat ein Agent eine Rückfrage, gehört sie als Kommentar an die Aufgabe — nicht in einen separaten Chat.

Rückfrage an den Menschen

Ich kann die Countdown-Karte in drei Richtungen ausarbeiten, benötige aber eine Entscheidung:

Option 1

Sehr minimalistische Karte mit viel Weißraum und klarer Typografie.

Option 2

Dunkle Gradient-Karte mit Glassmorphism, großer Countdown-Zahl und hochwertiger App-Ästhetik.

Option 3

Emotionale Karte mit Kategorie-Icon, leichter Illustration und stärker persönlicher Wirkung.

Meine Empfehlung

Für V1 empfehle ich Option 2 mit einzelnen Elementen aus Option 3, weil ChronoX dadurch hochwertig und emotional wirkt, ohne überladen zu werden.

Bitte entscheiden

Welche Richtung soll ich weiter konkretisieren?

Du antwortest dann ebenfalls als Kommentar oder durch Update der Aufgabe:

Entscheidung: Option 2, aber mit dem Kategorie-Icon aus Option 3.

Bitte konkretisiere die Gradient-/Glassmorphism-Variante.

Für V1 bitte bewusst einfach halten:

- Event-Titel oben links
- Kategorie-Icon oben rechts
- große Countdown-Zahl im Zentrum
- Ziel-Datum unten
- kein Fortschrittsbalken in V1

9.2.3 @mention

Ein @mention ist der direkteste Ersatz für ein „Kannst Du mal draufschauen?“ — aber nur dann brauchbar, wenn er Kontext, Ziel und erwartete Ausgabe enthält.

@ArchitectAgent Bitte prüfe die vorgeschlagene lokale Speicherung für ChronoX.

Fokus:

- passt die Lösung zu Kotlin Multiplatform?
- ist sie für Android und iOS geeignet?
- bleibt die App in V1 backendfrei?
- welche Risiken siehst Du bei Migrationen?
- bitte noch keinen Code ändern.

Was *nicht* funktioniert:

```
@ArchitectAgent thoughts?
```

Was funktioniert:

```
@ArchitectAgent Bitte prüfe die technische Option B für V1.  
Erwartete Ausgabe: kurze Bewertung mit Risiken, Empfehlung und möglichen Blockern.  
Noch keine Umsetzung.
```

9.2.4 Handoff / Übergabe

Paperclip selbst kennt keine eigene „Handoff“-Entity — das hier vorgestellte Format ist eine **Konvention**, die auf den Paperclip-Primitiven Task-Zuweisung, Kommentar und `@mention` aufsetzt. Der Vorteil: ein einheitliches, gut lesbares Übergabeformat, das spätere Leser nicht aus dem Kommentarverlauf rekonstruieren müssen.

Wenn ein Agent seine Arbeit weiterreicht, sollte er nicht einfach „fertig“ schreiben, sondern einen solchen strukturierten Handoff erzeugen.

Beispiel: Product Owner Agent an Architect Agent:

```
## Handoff an Architect Agent  
  
### Ausgangslage  
Der Mensch möchte für ChronoX eine lokale Speicherung der Countdown-Ereignisse  
definieren.  
  
### Produktentscheidung  
ChronoX soll in V1 vollständig ohne Backend funktionieren.  
Alle Countdowns werden lokal auf dem Gerät gespeichert.  
  
### Akzeptanzkriterien  
– Nutzer kann Ereignisse erstellen, bearbeiten und löschen.  
– Ereignisse bleiben nach App-Neustart erhalten.  
– Speicherung funktioniert auf Android und iOS.  
– Die Lösung bleibt für spätere Synchronisierung erweiterbar.  
  
### Nicht-Ziele  
– kein Backend in V1  
– keine Benutzerkonten  
– keine Cloud-Synchronisierung  
– kein Team-Sharing  
– keine komplexe Konfliktlösung  
  
### Bitte prüfen  
– technische Lösungsoptionen für Kotlin Multiplatform  
– Architekturplatzierung  
– Datenmodell  
– Migrationsstrategie  
– Risiken  
– V1-Empfehlung  
  
### Änderungsrechte  
Noch keinen Code ändern.
```

Der Vorteil gegenüber freier Agentenkommunikation: Jeder spätere Leser versteht, *warum* der nächste Agent aktiviert wurde.

9.2.5 Review Request

Paperclip hat eingebaute Review- und Approval-Stages als Governance-Schritt — das folgende Markdown-Template ist dagegen eine **Konvention**, die diesen Stages eine einheitliche Form gibt. Du kannst es auch als regulären Task an den Review Agent verwenden, wenn Du keine Approval-Stage brauchst.

```
## Review Request

### Ticket
Countdown-Karte für ChronoX implementieren

### Umsetzung
Ich habe die Event-Countdown-Karte gemäß Design-Spezifikation umgesetzt.

### Geänderte Dateien
- `CountdownCard.kt`
- `CountdownCardPreview.kt`
- `CountdownFormatter.kt`
- `CountdownFormatterTest.kt`

### Ausgeführte Tests
- Unit Tests für CountdownFormatter
- Compose Preview geprüft
- manuelle Prüfung für kurze und lange Event-Titel

### Bekannte Einschränkungen
- Fortschrittsbalken ist bewusst noch nicht enthalten.
- Animationen sind für spätere Versionen vorgesehen.

### Bitte prüfen
- Architekturkonformität
- Lesbarkeit der Compose-Komponente
- korrekte Countdown-Berechnung
- Design-Konformität zur Spezifikation
- Testabdeckung

### Gewünschtes Ergebnis
Review mit Blocker, Should Fix, Nice to Have und Freigabeempfehlung.
```

9.2.6 Approval Request

Wenn eine Entscheidung außerhalb des Agenten-Mandats liegt, erzeugt er einen Approval Request. Paperclip unterstützt Approvals als eigene Governance-Stage; das Markdown-Format unten ist die **Konvention** dafür, wie der Agent die zu treffende Entscheidung sauber präsentiert:

```
## Approval Request

### Entscheidung erforderlich
Soll ChronoX in V1 nur lokale Speicherung verwenden oder bereits eine optionale Cloud-Synchronisierung vorbereiten?

### Option A: rein lokale Speicherung
Vorteile: deutlich einfacher, keine Benutzerkonten nötig, keine Backend-Kosten, schneller V1-fähig.
Nachteile: keine Synchronisierung zwischen Geräten, Backup hängt vom Gerät bzw. System-Backup ab.

### Option B: lokale Speicherung plus vorbereitete Sync-Abstraktion
Vorteile: spätere Cloud-Synchronisierung einfacher, Architektur von Anfang an erweiterbarer.
Nachteile: höherer Aufwand in V1, Gefahr von Overengineering, mehr technische Entscheidungen nötig.

### Empfehlung
Option A für V1, aber mit sauberer Repository-Abstraktion, damit spätere Synchronisierung möglich bleibt.

### Benötigte Freigabe
```

Bitte Option A oder B bestätigen.
Ohne Entscheidung wird keine Implementierung gestartet.

10. Task-Modes: Interaktionsmodus pro Aufgabe festlegen

Jeder Task sollte einen expliziten Interaktionsmodus haben. Das ist die einzige Best Practice, die meine Paperclip-Arbeit am stärksten verändert hat.

Der Modus beantwortet eine simple Frage: Soll der Agent erst Vorschläge machen, iterativ mit Dir arbeiten oder direkt ausführen?

10.1 Grundprinzip

Jeder Task beginnt mit einer Zeile:

```
MODE: PROPOSE      → 2–3 Optionen erarbeiten, auf Auswahl warten
MODE: ITERATIVE    → erste Version, Feedback, nächste Iteration
MODE: EXECUTE      → direkt umsetzen, Scope ist klar
```

MODE ersetzt kein Chatfenster — er macht die taskbasierte Kommunikation präzise. Der Modus sollte auf den ersten Blick sichtbar sein, sonst muss jeder spätere Leser den Kontext rekonstruieren.

10.2 **MODE: PROPOSE**

Geeignet, wenn unklar ist, wie ein Feature, Design, eine Architekturentscheidung oder ein Workflow aussehen soll.

Regeln:

- 2-3 klar unterschiedliche Optionen, nummeriert als Option A/B/C
- jede Option in 3-5 Sätzen beschrieben
- Vor- und Nachteile pro Option
- abschließende Empfehlung
- keine Codeänderungen
- Stoppen mit **AWAITING_SELECTION**: Bitte wähle A, B oder C oder kombiniere Teile der Optionen.

Beispiel für ChronoX:

```
MODE: PROPOSE

TASK:
Gestalte die Countdown-Karte für ein Ereignis in ChronoX.

KONTEXT:
ChronoX ist eine visuell starke Countdown-App für persönliche Ereignisse, Meilensteine und Ziele.
Die App soll modern, emotional und hochwertig wirken.

ZIEL:
Bitte entwickle 3 unterschiedliche visuelle Richtungen für die Event-Countdown-Karte.

ERWARTETE AUSGABE:
– Option A: minimalistisch
– Option B: stark visuell / Gradient / Glassmorphism
– Option C: emotional / Kategorie-Icons / Illustrationselemente

EINSCHRÄNKUNGEN:
– Noch keinen Code ändern.
– Keine finale Designentscheidung treffen.
– Bitte nur Spezifikation, keine Implementierung.

AWAITING_SELECTION:
Am Ende bitte auf meine Auswahl warten.
```

Mögliche Antwort des Agenten:

Option A: Minimalistische Countdown-Karte

Die Karte konzentriert sich auf den Event-Titel, das Datum und die verbleibende Zeit. Der Hintergrund bleibt ruhig, mit klarer Typografie und viel Weißraum.

Vorteile: sehr übersichtlich, leicht umzusetzen, gute Lesbarkeit.

Nachteile: wenig emotionale Wirkung, visuell weniger eigenständig.

Option B: Glassmorphism Dark Gradient

Dunkler Verlaufshintergrund, leichte Transparenzflächen, große Countdown-Zahlen. ChronoX wirkt dadurch moderner und stärker wie eine hochwertige Lifestyle-App.

Vorteile: starke visuelle Wirkung, gutes App-Store-Potenzial, passt zu persönlichen Meilensteinen.

Nachteile: höherer Designaufwand, Kontrast und Lesbarkeit müssen sorgfältig geprüft werden.

Option C: Illustrativ mit Kategorie-Icons

Klares Layout, ergänzt um Kategorie-Icons oder kleine Illustrationen – Reise, Geburtstag, Prüfung, Projektziel. Dadurch wird jeder Countdown persönlicher und schneller erfassbar.

Vorteile: emotional und zugänglich, gute Wiedererkennbarkeit, Kategorien werden visuell unterstützt.

Nachteile: Icon-System muss konsistent gepflegt werden, Gefahr von zu viel visueller Unruhe.

AWAITING_SELECTION: Bitte wähle A, B oder C oder kombiniere Teile der Optionen.

10.3 MODE: ITERATIVE

Passend, wenn die Richtung steht, aber Details noch gemeinsam ausgearbeitet werden müssen — typisch für UI-Design, UX-Flows, Architekturvarianten oder Produkttexte.

Regeln:

- erste Version liefern, dann auf Feedback warten
- jede Runde nummerieren
- maximal fünf Iterationen, danach eskalieren
- pro Runde klar nennen, was sich geändert hat
- Stoppen mit **ITERATION_[N] COMPLETE. Feedback?**

Beispiel:

MODE: ITERATIVE

TASK:

Konkretisiere die Countdown-Karte auf Basis von Option B, aber übernimm den Icon-Ansatz aus Option C.

KONTEXT:

Gewählte Richtung:

- Glassmorphism Dark Gradient
- große Countdown-Zahlen
- zusätzlich Kategorie-Icon oben rechts
- emotional, aber nicht verspielt

ZIEL:

Erstelle eine konkrete Design-Spezifikation für die Karte.

ERWARTETE AUSGABE:

- Layout, Typografie, Farben
- Zustände, Interaktionen, leere Zustände
- spätere Ausbaustufen

EINSCHRÄNKUNGEN:

- Noch keinen Code ändern.
- Erst Spezifikation.
- Nach jeder Iteration auf Feedback warten.

Antwort:

ITERATION_1 – Countdown Card Specification

Grundidee

Die Event-Karte erhält einen dunklen, hochwertigen Verlaufshintergrund mit leichtem Glassmorphism-Effekt. Im Zentrum steht die verbleibende Zeit, z. B. „42 Tage“. Das Kategorie-Icon sitzt oben rechts in einer dezenten runden Fläche.

Layout

- oben links: Event-Titel
- oben rechts: Kategorie-Icon
- Mitte: große Countdown-Zahl
- darunter: Einheit (Tage / Stunden / Minuten)
- unten: Datum und optionaler Fortschrittsindikator

Visuelle Richtung

Dunkler Gradient, weiche Schatten, runde Ecken, hohe Kontraste, ruhige App-Ästhetik.

Offene Designfrage

Soll der Fortschrittsindikator bereits in V1 enthalten sein oder erst später?

ITERATION_1 COMPLETE. Feedback?

Menschliche Antwort:

Feedback zu Iteration 1:

- Fortschrittsindikator erst später.
- In V1 bitte nur Titel, Kategorie-Icon, Countdown-Zahl, Datum.
- Die Karte darf emotional wirken, aber nicht nach Gaming aussehen.
- Bitte konkretisiere daraus eine V1-Spezifikation.

10.4 MODE: EXECUTE

Nur einsetzen, wenn Ziel klar ist, Akzeptanzkriterien vorliegen, der Scope begrenzt ist, relevante Design- oder Architekturentscheidungen getroffen wurden und keine blockierenden Fragen mehr offen sind.

Regeln:

- Aufgabe direkt umsetzen, strikt im Scope bleiben
- geänderte Dateien dokumentieren
- relevante Tests ausführen
- Risiken und offene Punkte melden
- bei unerwarteten Produkt-, UX- oder Architekturfragen stoppen und **HUMAN_INPUT_REQUIRED** schreiben

Beispiel:

MODE: EXECUTE

TASK:

Implementiere die ChronoX Countdown Card gemäß Design-Spezifikation.

KONTEXT:

Die abgestimmte Spezifikation liegt in:
 `docs/design/components/countdown-card.md`

AKZEPTANZKRITERIEN:

- Event-Titel wird angezeigt.
- Kategorie-Icon wird angezeigt.
- verbleibende Zeit wird groß dargestellt.
- Ziel-Datum wird angezeigt.
- Karte ist responsiv.
- Dark-Gradient-Stil entspricht der Design-Spezifikation.
- Keine Fortschrittsanzeige in V1.

NICHT-ZIELE:

- keine Animationen
- kein Swipe-Verhalten
- kein komplexer Detail-Screen
- keine Änderung am Datenmodell

REGELN:

- Halte Dich an AGENTS.md, DESIGN.md und `docs/design/components/countdown-card.md`.
- Ändere nur Dateien, die für diese Karte notwendig sind.
- Ergänze Preview/Testfälle, falls im Projekt üblich.
- Am Ende Abschlussbericht mit Dateien, Tests und Risiken liefern.

10.5 Modes in System-Prompts verankern

Die Modi sollten nicht nur im Tutorial leben, sondern direkt in den Rollenbeschreibungen der Agenten stehen. So musst Du sie nicht in jedem Task neu erklären — der Agent kennt sie bereits.

Interaktionsmodi

Jeder Task kann einen expliziten `MODE` enthalten. Der Agent muss diesen Modus strikt beachten.

MODE: PROPOSE

- Erstelle 2-3 klar unterschiedliche Optionen.
- Nummeriere sie als Option A, Option B, Option C.
- Beschreibe jede Option in 3-5 Sätzen.
- Nenne Vorteile und Nachteile.
- Gib eine Empfehlung.
- Führe nichts aus, ändere keine Dateien.
- Stoppe mit:

```text

AWAITING\_SELECTION: Bitte wähle A, B oder C oder kombiniere Teile der Optionen.

```

MODE: ITERATIVE

- Liefere eine erste Version.
- Warte auf Feedback.
- Zähle Iterationen mit, max. 5.
- Nach 5 Iterationen eskaliere an den CEO/Lead Agent oder an den Menschen.
- Ändere keine Dateien, außer der Task erlaubt es ausdrücklich.
- Stoppe nach jeder Iteration mit:

```text

ITERATION\_[N] COMPLETE. Feedback?

```

MODE: EXECUTE

- Führe die Aufgabe direkt aus.
- Bleibe strikt im Scope.
- Dokumentiere Änderungen, führe Tests aus, melde Risiken.

– Bei blockierenden Fragen stoppen und schreiben:

```
```text  
HUMAN_INPUT_REQUIRED
```
```

11. Direkte Agentenzuweisung vs. Delegation über CEO/Lead Agent

Es lohnt sich, bewusst zu entscheiden, ob ein Task direkt an einen Spezialisten geht oder über einen koordinierenden Agenten läuft.

„Direkt mit einem Agenten arbeiten“ heißt in Paperclip nicht zwingend Chat. Gemeint ist: Du erstellst oder kommentierst einen Task direkt für diesen Agenten.

11.1 Direkte Zuweisung

Geeignet für isolierte Exploration, Designvarianten, Textentwürfe, kleine Spezifikationen oder Spezialistenfragen.

```
Mensch -> UX/UI Agent
MODE: PROPOSE
TASK: Erstelle 3 visuelle Richtungen für die Countdown-Karte.
```

Vorteil: wenig Koordinationsaufwand, schnelle Exploration. Nachteil: Ergebnisse müssen danach bewusst in gemeinsame Artefakte überführt werden, sonst kennen andere Agenten sie nicht.

11.2 Delegation über CEO/Lead Agent

Geeignet für Features mit mehreren beteiligten Agenten, Arbeit mit Abhängigkeiten, Umsetzung nach abgestimmter Spezifikation, Architekturentscheidungen, Review- und QA-Prozesse.

```
Mensch -> CEO/Lead Agent
MODE: EXECUTE
TASK: Setze die abgestimmte Countdown Card um.
INPUT: docs/design/components/countdown-card.md
```

Der CEO/Lead Agent koordiniert dann:

```
CEO/Lead -> Engineer Agent
CEO/Lead -> Review Agent
CEO/Lead -> Documentation Agent
```

Vorteil: bessere Koordination, weniger Kontextverlust, Review und Doku werden mitgedacht. Nachteil: mehr Prozess — für reine Exploration manchmal überdimensioniert.

11.3 Praktische Empfehlung für ChronoX

| Situation | Modus | Zuweisung |
|--|------------------------|-------------------------------|
| „Ich weiß nicht, wie X aussehen soll“ | PROPOSE | direkt an UX/UI Agent |
| „Ich mag die Richtung, Details klären“ | ITERATIVE | direkt an UX/UI Agent |
| „Design ist abgestimmt, bitte Spezifikation schreiben“ | ITERATIVE oder EXECUTE | UX/UI Agent |
| „Feature ist klar definiert, bitte bauen“ | EXECUTE | über CEO/Lead → Engineer |
| „Design fertig, Engineer soll es umsetzen“ | EXECUTE | über CEO/Lead mit Design-Spec |
| „Architektur ist unklar“ | PROPOSE | Architect Agent |
| „Code ist fertig, bitte prüfen“ | EXECUTE Review-Modus | Review Agent |
| „Feature ist fertig, bitte dokumentieren“ | EXECUTE | Documentation Agent |

11.4 Gemeinsame Design-Spezifikationen

Eine Entscheidung, die im Ping-Pong mit dem UX/UI Agent entsteht, sollte nicht im Task-Kommentar versanden. Sie gehört in eine zitierfähige Spezifikation:

```
docs/design/components/countdown-card.md
docs/design/components/event-list.md
docs/design/components/event-detail.md
docs/design/flows/create-countdown.md
```

Der Engineer Agent bekommt damit nicht einen Kommentarverlauf, sondern ein abgestimmtes Dokument.

Empfohlene Repo-Struktur (konsistent mit Kapitel 4 — Top-Level-Dateien im Root, Detail-Artefakte unter `docs/`):

```
/VISION.md
/AGENTS.md
/WORKFLOW.md
/TASKS.md
/HEARTBEAT.md
/DESIGN.md
/ARCHITECTURE.md
/DECISIONS.md
/SECURITY.md
docs/
  design/
    components/
      countdown-card.md
      event-list.md
    flows/
      create-countdown.md
  decisions/
    2026-04-30-countdown-card-design.md
```

Warum `docs/design/...` und nicht `shared/design/...`? `shared/` klingt zu schnell nach Runtime-Code. Für Agenten-Artefakte ist `docs/` eindeutiger.

Warum die Top-Level-Dateien im Root? Weil das offene `AGENTS.md`-Format genau das vorsieht und weil es für menschliche Leser sofort sichtbar ist, ohne erst in einen Ordner zu klicken. `DECISIONS.md` ist die kuratierte Übersicht; einzelne ADRs liegen in `docs/decisions/` mit Datum im Dateinamen.

12. Wie kommuniziert der Mensch mit Agenten?

Du kommunizierst über neue Aufgaben, Kommentare, Antworten auf Rückfragen, Freigaben, Ablehnungen, Änderungswünsche und Priorisierungen.

Der wichtigste Unterschied zu klassischem Prompting: Du gibst Agenten nicht nur Aufgaben, sondern Arbeitsmodi. Der Modus gehört maschinenlesbar an den Anfang des Tasks (siehe Kapitel 10) — der Agent muss dann nicht mehr raten, ob er explorieren, iterieren oder umsetzen soll.

Statt:

```
Baue mir das Feature.
```

Besser:

```
Analysiere zuerst das Feature.  
Stelle offene Fragen.  
Mache 3 Umsetzungsvorschläge.  
Bewerte Aufwand, Risiko und UX.  
Empfehl eine Richtung.  
Ändere noch keinen Code.
```

12.1 Wenn ein Agent Dir eine Frage stellen muss

Nicht als freie Chatfrage, sondern als blockierende Rückfrage im Task — mit festem Format:

```
## Human Input Required  
  
### Status  
Blocked  
  
### Warum ich blockiert bin  
[Kurze Erklärung]  
  
### Entscheidung, die benötigt wird  
[Konkrete Frage]  
  
### Optionen  
1. [Option A]  
2. [Option B]  
3. [Option C]  
  
### Empfehlung des Agenten  
[Empfehlung mit Begründung]  
  
### Was nach der Entscheidung passiert  
[Konkreter nächster Schritt]
```

Beispiel:

```
## Human Input Required  
  
### Status  
Blocked  
  
### Warum ich blockiert bin  
Der Create-Countdown-Flow kann entweder extrem reduziert oder emotionaler mit  
Kategorieauswahl und Live-Vorschau gestaltet werden. Beide Richtungen führen zu  
unterschiedlichen Screens und Akzeptanzkriterien.  
  
### Entscheidung, die benötigt wird  
Welche Richtung soll für V1 im Vordergrund stehen?
```

Optionen

1. Minimal: Titel und Ziel-Datum, danach direkt speichern
2. Emotional: Titel, Ziel-Datum, Kategorie und Live-Vorschau der Countdown-Karte
3. Erweitert: zusätzlich Templates, Farben und Erinnerungen

Empfehlung des Agenten

Ich empfehle Option 2, weil sie ChronoXs visuelle Stärke unterstützt, aber noch V1-tauglich bleibt.

Was nach der Entscheidung passiert

Nach Deiner Antwort erstelle ich ein konkretes UI-Konzept mit Screens, States und Akzeptanzkriterien.

Die menschliche Antwort als Kommentar:

Entscheidung: Option 2.

Bitte aber bewusst V1-tauglich:

- Live-Vorschau ja
- Templates noch nicht
- Erinnerungen noch nicht
- Kategorieauswahl nur mit wenigen festen Kategorien

12.2 Vorlagen für typische Situationen

Wenn Du noch nicht weißt, wie ein Feature aussehen soll

Task: Explorationsauftrag für Feature X

Ziel

Ich bin mir noch nicht sicher, wie dieses Feature aussehen soll.
Bitte arbeite nicht direkt an der Umsetzung, sondern entwickle zuerst Varianten.

Feature-Idee

[Beschreibung]

Bitte liefere 3 Varianten

1. Minimalistische V1
2. Gute Produktvariante
3. Ambitionierte Premium-Variante

Pro Variante bitte beschreiben

- fachliche Idee, UX-Ablauf, technische Konsequenzen
- Aufwand, Risiken
- Vorteile, Nachteile

Erwartetes Ergebnis

Eine Empfehlung, aber keine finale Entscheidung.

Änderungsrechte

Keine Codeänderungen, keine Architekturänderungen.

Danach

Ich wähle eine Richtung aus oder kombiniere mehrere Varianten.

Wenn Du Designvarianten sehen möchtest

Task: Designvarianten für [Feature]

Rolle

UX/UI Agent

Ziel

Ich möchte für folgendes Feature 3 Designrichtungen sehen:

[Feature-Beschreibung]

Bitte beschreibe je Richtung

- Layout-Idee, visuelle Wirkung
- Nutzerführung, Komponenten
- Vorteile, Nachteile
- wann diese Richtung geeignet ist

Einschränkungen

Noch keinen Code schreiben.

Erwartung

Die Varianten sollen so konkret sein, dass ich danach eine Richtung auswählen kann.

Wenn Du eine Richtung auswählst

Entscheidung: Ich wähle Variante 2.

Bitte konkretisiere diese Variante jetzt zu einem umsetzbaren UI-Konzept.

Erstelle:

- Screen-Struktur, Komponentenliste, Zustände
- Interaktionen, leere Zustände, Fehlerzustände
- responsives Verhalten, Akzeptanzkriterien

Noch keinen Code ändern.

Wenn aus dem Konzept ein Ticket werden soll

Bitte erstelle aus diesem Konzept ein implementierbares Ticket.

Das Ticket soll enthalten:

- Ziel, Nutzerwert, fachliche Beschreibung
- Akzeptanzkriterien, technische Hinweise
- Nicht-Ziele, Testfälle, offene Fragen
- Definition of Done

Bitte so formulieren, dass ein Engineer Agent es direkt umsetzen kann.

Wenn der Engineer loslegen soll

Bitte arbeite im Umsetzungsmodus.

Setze folgendes Ticket um:

[Ticket]

Regeln:

- Halte Dich an VISION.md, AGENTS.md, ARCHITECTURE.md und DESIGN.md.
- Ändere nur Dateien, die für dieses Ticket notwendig sind.
- Ergänze oder aktualisiere passende Tests.
- Führe relevante Checks aus.
- Dokumentiere am Ende die Änderungen.

Wenn während der Umsetzung eine Produkt- oder Architekturentscheidung offen ist, stoppe und frage nach.

13. Wie kommunizieren Agenten untereinander?

Agenten sollten nicht über „interne Gedanken“ kommunizieren, sondern über sichtbare Artefakte. Eine Übergabe gehört dokumentiert, nicht erraten.

Die typischen Formen sind: neuer Task, Kommentar mit @mention, Handoff-Kommentar, Review Request, Work Product, Approval Request, Aktualisierung von `TASKS.md` / `DECISIONS.md` / `ARCHITECTURE.md`. Davon sind **Tasks**, **Kommentare**, **@mentions**, **Work Products** und **Approval-Stages** echte Paperclip-Primitive — *Handoff* und *Review Request* sind Konventionen, die auf Kommentaren mit klarer Struktur aufsetzen (siehe 9.2.4 und 9.2.5).

13.1 Handoff-Format

```
## Handoff an: [Agent]

### Kontext
Was ist bisher bekannt?

### Ziel
Was soll der Agent erreichen?

### Nicht-Ziele
Was soll ausdrücklich nicht gemacht werden?

### Relevante Dateien
Welche Dateien soll der Agent lesen?

### Erwartete Ausgabe
Was soll konkret zurückkommen?

### Entscheidungsspielraum
Darf der Agent entscheiden oder nur vorschlagen?

### Änderungsrechte
Darf der Agent Code ändern? Ja / Nein

### Eskalation
Wann muss der Mensch gefragt werden?
```

13.2 Beispiel: UX Agent an Engineer Agent

```
## Handoff an Engineer Agent

### Ziel
Bitte implementiere die gewählte Countdown-Karten-Variante.

### Gewählte Richtung
Dark-Gradient Countdown Card mit kompaktem Kategorie-Icon.

### Relevante UX-Entscheidungen
- Die verbleibende Zeit steht im Mittelpunkt.
- Event-Titel und Ziel-Datum bleiben klar lesbar.
- Kategorie-Icon unterstützt die emotionale Wirkung.
- Kein Fortschrittsbalken in V1.
- Keine Animationen in diesem Ticket.

### Komponenten
`CountdownCard`, `CountdownValue`, `CategoryIconBadge`, `TargetDateLabel`

### States
- Event liegt in der Zukunft
- Event ist heute
```

- Event ist abgelaufen
- Event-Titel ist sehr lang
- Kategorie fehlt oder ist neutral

Nicht-Ziele

keine Swipe-Actions, kein komplexer Detail-Screen, keine Widget-Umsetzung, keine Theme-Auswahl in diesem Ticket

Akzeptanzkriterien

- Nutzer sieht Event-Titel, Kategorie-Icon, Countdown und Ziel-Datum.
- Countdown ist groß und schnell erfassbar.
- Karte bleibt auch bei langen Titeln lesbar.
- Layout funktioniert auf Android und iOS.

Bitte umsetzen

Änderungsrechte: Ja, im Scope dieses Tickets.

13.3 Fragen müssen entscheidbar sein

Ein Agent sollte Dir keine schwammigen Fragen stellen.

Schlecht:

Was denkst du?

Besser:

Bitte entscheide zwischen:

- A: minimalistische Countdown-Karte
- B: Dark-Gradient Countdown-Karte
- C: illustrative Countdown-Karte mit Kategorie-Icons

Meine Empfehlung: B mit Icon-Elementen aus C.

Antworte bitte mit A, B oder C plus optionalen Anpassungen.

Je klarer die Antwortstruktur, desto leichter können Agenten sie weiterverarbeiten — und desto weniger Fehlinterpretationen schleichen sich ein.

14. Ping-Pong-Arbeit ohne Chatfenster

Bei Design, UX, Architektur und Produktdetails liefert ein Ping-Pong-Modus die besten Ergebnisse: Der Agent arbeitet nicht autonom bis zum Ende, sondern in kleinen Schritten mit Zwischenergebnissen.

In Paperclip läuft das nicht über Chat, sondern über kleine Aufgaben, Kommentare, Statuswechsel, blockierende Rückfragen und Folgetasks.

14.1 Beispielablauf

1. Mensch beschreibt grobe Idee.
2. Agent liefert Varianten.
3. Mensch wählt Richtung.
4. Agent konkretisiert.
5. Mensch korrigiert.
6. Agent erstellt Ticket.
7. Mensch gibt frei.
8. Engineer Agent implementiert.
9. Review Agent prüft.
10. Mensch entscheidet final.

In Aktion sieht das so aus:

Aufgabe 1 — Varianten erstellen. Der UX-Agent liefert als Kommentar drei Vorschläge mit Empfehlung. Status: **Waiting for Human**.

Antwort. „Ich wähle Variante B, aber mit dem Kategorie-Icon-Ansatz aus Variante C. Bitte konkretisieren.“

Aufgabe 2 — Konkretisierung. Der Agent liefert Screen-Struktur, Komponenten, States. Eine Designentscheidung bleibt offen. Status erneut: **Waiting for Human**.

Antwort. „Fortschrittsbalken bitte erst später. Danach bitte daraus ein implementierbares Ticket erstellen.“

Folgetask. „Countdown Card im Dark-Gradient-Stil implementieren“, zugewiesen an den Engineer Agent.

Das ist Ping-Pong — nur eben ticketbasiert statt chatbasiert.

14.2 Prompt für Ping-Pong-Modus

Bitte arbeite mit mir im Ping-Pong-Modus.

Wichtig:

- Mache kleine Schritte.
- Liefere zuerst einen Vorschlag.
- Warte nach jedem größeren Entscheidungspunkt auf mein Feedback.
- Stelle gezielte Rückfragen, wenn eine Entscheidung notwendig ist.
- Triff keine weitreichenden Produkt-, Design- oder Architekturentscheidungen ohne Rückmeldung.
- Ändere keinen Code, bevor ich ausdrücklich "umsetzen" sage.

Thema:

[Beschreibung]

Beim Design lohnt sich oft ein zweistufiger Einstieg:

Bitte arbeite als UX/UI Agent im Ping-Pong-Modus.

Ziel: Wir wollen eine gute Darstellung für [Feature] finden.

Bitte starte mit 3 groben Richtungen.

Noch keine Details, kein Code.

Ich möchte danach eine Richtung auswählen oder Teile kombinieren.

Und dann iterativ:

Variante 1 gefällt mir vom Aufbau.

Variante 3 gefällt mir visuell besser.

Bitte kombiniere beides und erstelle daraus ein konkreteres UI-Konzept.

Das ist gut, aber reduziere die Komplexität für V1.

Bitte mache daraus eine minimalistische erste Version mit späteren Ausbaustufen.

15. Arbeitsmodi als zentrales Best Practice

Die Modi aus Kapitel 10 sind die Schreibweise für Tasks. In `AGENTS.md` und Rollenbeschreibungen tauchen sie noch einmal als generische *Arbeitsmodi* auf — etwas breiter gefasst, weil sie auch ohne expliziten `MODE`-Tag gelten.

Analysemodus — verstehen, Risiken erkennen, Optionen vorbereiten. Lesen, analysieren, erklären, Rückfragen stellen. Keine Codeänderungen, keine finalen Entscheidungen.

Vorschlagsmodus — 2-3 Varianten mit Vor-/Nachteilen, Aufwand, Risiken, Empfehlung. Keine Implementierung.

Spezifikationsmodus — aus einer Idee ein klares Ticket machen: User Story, Akzeptanzkriterien, Testfälle, Nicht-Ziele, Definition of Done.

Umsetzungsmodus — Code oder Artefakte erzeugen, im Scope. Änderungen dokumentieren, Tests nennen, Risiken melden.

Reviewmodus — Funktion, Architektur, Tests, Security, Wartbarkeit, UX-Konformität prüfen.

Dokumentationsmodus — README, `DECISIONS.md`, `ARCHITECTURE.md`, CHANGELOG, Nutzeranleitungen aktualisieren.

In der Praxis genügen die drei Task-Modi aus Kapitel 10 für 90 % der Aufgaben. Die feinere Aufteilung hilft vor allem dann, wenn Du Rollen sauber definieren willst.

16. Typischer Workflow: von Idee zu fertigem Feature

Ein Beispiel von der losen Idee bis zum gemergten Feature — alle neun Schritte. Im Unterschied zu den vorherigen Kapiteln zeige ich hier nicht nur, *was* gesagt wird, sondern auch *wo es konkret passiert*: welcher Task wird angelegt, wer antwortet wo, und wie geht es zum nächsten Schritt weiter.

Die Grundmechanik in einem Absatz

Alles in Paperclip läuft über **Tasks** und ihre **Kommentare**. Eine Idee wird zu einem Task. Der Task wird einem Agenten zugewiesen. Der Agent antwortet im selben Task — entweder als Kommentar, als Work Product (eine Datei) oder durch ein strukturiertes Handoff. Folgearbeit entsteht entweder als neuer Task oder als Handoff-Kommentar mit `@mention`. Der Status zeigt jederzeit, wo Du im Prozess stehst — die Werte (`Backlog`, `Ready`, `In Progress`, `Waiting for Human`, `Blocked`, `In Review`, `Done`, `Deferred`) sind keine Paperclip-Vorgabe, sondern eine Projekt-Konvention, die in `WORKFLOW.md` §13 verbindlich definiert wird. Wichtige Entscheidungen wandern parallel in `DECISIONS.md`, abgestimmte Spezifikationen in `docs/design/...`.

Mit diesem Bild im Kopf jetzt der konkrete Ablauf.

Schritt 1: Idee als Task anlegen

Wo: Du erstellst einen neuen Task in Paperclip. Titel z. B. „Create-Countdown-Flow konzipieren“. Assigned to: **Product Owner Agent**. Status: `Ready` (es ist klar, was zu tun ist und wer es tun soll). Im Task-Body steht die rohe Idee — bewusst noch ohne Lösungsansatz:

```
## Task: Create-Countdown-Flow konzipieren

### Assigned to
Product Owner Agent

### Idee
Nutzer sollen in ChronoX ein neues Countdown-Ereignis anlegen können.
Ich bin mir aber noch nicht sicher, wie reduziert oder emotional der Create-Flow
aussehen soll.

### Erwartung
Bitte erst analysieren und Varianten vorschlagen. Noch keine Umsetzung.
```

Übergang: Beim nächsten Trigger / Heartbeat nimmt der PO-Agent den Task auf.

Schritt 2: Product Owner Agent analysiert

Wo: Der PO-Agent liest den Task. Statt direkt loszulegen, ergänzt **Du** im Task einen Auftrag mit explizitem Modus — entweder als Kommentar oder als Aktualisierung des Task-Bodys:

```
MODE: PROPOSE

Bitte analysiere die Feature-Idee aus diesem Task.
Erstelle 3 Produktvarianten:
1. Minimaler V1-Flow
2. Emotionaler App-Flow mit Kategorieauswahl
3. Ambitionierter Flow mit Templates und Vorschau

Pro Variante: Nutzen, Aufwand, Risiken, UX-Komplexität.
Empfehlung am Ende.
Keine Umsetzung.
```

Was zurückkommt: Der Agent antwortet als Kommentar mit den drei Varianten und seiner Empfehlung. Status wechselt auf `Waiting for Human`.

Schritt 3: Mensch entscheidet

Wo: Du antwortest als Kommentar im selben Task. Keine neuen Tasks, kein Chat — die Entscheidung gehört zum Vorgang:

Entscheidung: Variante 2, aber ohne Templates.

Für V1 reicht:

- Titel
- Ziel-Datum
- optionale Ziel-Uhrzeit
- Kategorie
- Vorschau der Countdown-Karte

Übergang: Der PO-Agent erstellt jetzt einen **Handoff** an den UX/UI Agent — entweder als Kommentar mit `@UXAgent` oder als neuen Task „Create-Flow konkretisieren“. Ich bevorzuge den neuen Task, weil sich UX-Iterationen sonst mit der Produktdiskussion vermischen.

Schritt 4: UX/UI Agent konkretisiert

Wo: Neuer Task „Create-Countdown-Flow: UX-Konzept“, Assigned: **UX/UI Agent**. Im Body verweist Du auf den Vorgänger-Task und gibst den Auftrag:

MODE: ITERATIVE

Kontext: siehe Task #42 (Produktentscheidung Variante 2 ohne Templates).

Bitte konkretisiere den Create-Countdown-Flow:
Screen-Struktur, Eingabefelder, Zustände, Fehlerfälle, Vorschauverhalten,
Akzeptanzkriterien.

Noch keinen Code ändern.

Nach jeder Iteration auf Feedback warten.

Was zurückkommt: Der Agent liefert `ITERATION_1` als Kommentar. Du gibst Feedback, der Agent liefert `ITERATION_2`, und so weiter. Wenn das Konzept steht, schreibt der Agent es als Spezifikationsdatei nach `docs/design/flows/create-countdown.md` — ab da ist es zitierfähig.

Wer schreibt die Datei wirklich? Das hängt von der Wahl des Agent-Backends ab (Bring Your Own Agent — Kapitel 3.1). Ein Cursor- oder Claude-Code-Agent kann direkt ins Repo committen; ein reiner HTTP-Webhook-Agent liefert die Markdown-Inhalte als Work Product zurück, und der Documentation Agent (oder Du selbst) committet im nächsten Heartbeat. Wichtig ist nur, dass die Datei am Ende existiert — der Weg dorthin ist Setup-Frage, nicht Workflow-Frage.

Schritt 5: Architect Agent prüft

Wo: Parallel (nicht sequenziell) erstellst Du einen Task an den **Architect Agent**. Parallel deshalb, weil UX und Architektur unabhängig voneinander sind, solange das Datenmodell nicht streitig ist:

MODE: PROPOSE

Bitte prüfe das Feature aus Task #42 aus Architektursicht.
Zeige 2 technische Optionen für lokale Speicherung in Kotlin Multiplatform.
Bewerte Einfachheit, Wartbarkeit, Migrationen, spätere Sync-Fähigkeit.
Empfehl eine V1-Lösung.
Noch keinen Code ändern.

Was zurückkommt: Optionen als Kommentar, Empfehlung. Du entscheidest im Kommentar; die Entscheidung gehört anschließend in `DECISIONS.md` (Datum, Begründung, Konsequenzen). Das macht entweder der Documentation Agent bei seinem nächsten Heartbeat — er ist laut `AGENTS.md` für die Überführung von Task-Entscheidungen in Doku zuständig — oder Du selbst direkt nach der Entscheidung. Bei mir hat sich bewährt, kleine Entscheidungen sofort selbst einzutragen und nur die längeren ADRs dem Documentation Agent zu überlassen.

Schritt 6: Ticket erstellen

Wo: Wenn UX-Spezifikation und Architekturentscheidung beide stehen, beauftragst Du den PO-Agent (oder einen Lead Agent), aus beidem ein implementierbares Ticket zu bauen. Das passiert wieder als Task:

MODE: EXECUTE

Bitte erstelle ein implementierbares Ticket aus:

- docs/design/flows/create-countdown.md (UX-Konzept)
- DECISIONS.md Eintrag vom \$DATUM (Speicherentscheidung)

Ticket soll enthalten: Titel, Ziel, Akzeptanzkriterien, Nicht-Ziele, technische Hinweise, Testfälle, Definition of Done.

Was zurückkommt: Ein fertig formuliertes Ticket — entweder als neuer Task „Implementiere Create-Countdown-Flow“ oder als Markdown-Anhang, den Du als Task einstellst. Dieses Ticket ist die Grundlage für Schritt 7.

Schritt 7: Engineer Agent implementiert

Wo: Du weist das Ticket dem **Engineer Agent** zu (oder lässt es vom Lead Agent zuweisen). Status: **In Progress**.

MODE: EXECUTE

Bitte setze dieses Ticket um.

Halte Dich an AGENTS.md, DESIGN.md, ARCHITECTURE.md und docs/design/flows/create-countdown.md.

Führe relevante Tests aus.

Stoppe und schreibe HUMAN_INPUT_REQUIRED, falls eine ungeklärte Produkt-, UX- oder Architekturentscheidung auftaucht.

Was zurückkommt: Code-Änderungen plus Abschlussbericht im Task-Kommentar — geänderte Dateien, ausgeführte Tests, offene Risiken. Status: zunächst **In Progress**, danach **In Review** (sobald der Engineer den Review Request abgesetzt hat).

Schritt 8: Review Agent prüft

Wo: Der Engineer-Agent erzeugt einen **Review Request** (siehe Kapitel 9.2.5) — entweder als Kommentar mit @ReviewAgent im Implementierungs-Task oder als separaten Review-Task, der den Implementierungs-Task referenziert. Status während des Reviews: **In Review**.

MODE: EXECUTE

@ReviewAgent Bitte reviewe die Änderungen aus Task #58.

Prüfe Architektur, Tests, UX-Konformität, Fehlerfälle, Wartbarkeit.

Gib das Ergebnis als Blocker / Should Fix / Nice to Have / Freigabeempfehlung.

Was zurückkommt: Review-Kommentar mit den vier Kategorien und einer expliziten Freigabeempfehlung. Status wechselt von **In Review** auf **Waiting for Human** — Du bist jetzt am Zug, weil der Reviewer eine Empfehlung abgegeben hat, aber die finale Freigabe nicht selbst aussprechen darf.

Schritt 9: Mensch entscheidet final

Wo: Du antwortest im Task-Kommentar mit der finalen Entscheidung:

Entscheidung:

- Blocker 1 und 2 bitte beheben (zurück an Engineer Agent).
- Should-Fix 2 wandert in einen neuen Folgetask "Refactor CountdownFormatter".
- Nach Behebung der Blocker: Freigabe.

Übergang: Engineer-Agent behebt die Blocker (Mini-Loop von Schritt 7-9), danach setzt Du den Task auf **Done**, der Folgetask landet auf dem Board, und das Feature ist live.

Was an dieser Mechanik zentral ist

Jeder Schritt erzeugt ein nachvollziehbares Artefakt: einen Task-Kommentar, eine Spezifikationsdatei, einen DECISIONS.md -Eintrag oder Code. Niemand muss später raten, *warum* eine Entscheidung gefallen ist — sie steht in derselben Spur, in der sie gefällt wurde.

17. Best Practices für produktives Arbeiten mit Paperclip

Diese Punkte sind das Destillat aus eigenen Fehlversuchen.

17.1 Starte klein

Nicht sofort 20 Agenten anlegen. Lieber in Phasen wachsen:

```
Phase 1: Product Owner, Engineer, Reviewer
Phase 2: + Architect, QA Agent
Phase 3: + UX/UI, Documentation, DevOps
```

17.2 Gib jedem Agenten eine klare Identität

Generische Namen wie „Agent 1“ oder „Coding Bot“ verschwimmen schnell. Echte Namen plus Rolle bleiben hängen — und helfen mir selbst, zwischen den Verantwortlichkeiten nicht durcheinanderzukommen:

```
Mara – Product Owner Agent
Anton – Architecture Agent
Elena – Backend Engineer Agent
Nora – Review Agent
Mika – UX/UI Agent
```

17.3 Trenne Denken und Umsetzen

Einer der häufigsten Fehler: zu früh „bau das mal“. Viele schlechte KI-Ergebnisse entstehen nicht, weil der Agent „dumm“ ist, sondern weil er ohne Analyse direkt in die Umsetzung geschickt wurde. Erst denken, dann bauen — auch wenn es zwei Tasks statt einem braucht.

17.4 Definiere Nicht-Ziele

Ohne Nicht-Ziele neigen Agenten dazu, „hilfreich“ zu sein und zu viel zu bauen. Ein paar Stichpunkte reichen:

```
## Nicht-Ziele für V1

– kein Backend
– keine Benutzerkonten
– keine Cloud-Synchronisierung
– keine Social-Media-Funktionen
– keine komplexe Gamification
– keine Widget-Umsetzung in diesem ersten Ticket
```

17.5 Verwende Akzeptanzkriterien

Statt:

```
Baue Countdown-Erstellung.
```

Lieber:

```
## Akzeptanzkriterien

– Nutzer kann einen Countdown mit Titel und Ziel-Datum erstellen.
– Optional kann eine Ziel-Uhrzeit gesetzt werden.
– Optional kann eine Kategorie gewählt werden.
– Nach dem Speichern erscheint der Countdown in der Liste.
– Ungültige Eingaben erzeugen verständliche Fehlermeldungen.
– Es gibt Tests für gültige und ungültige Eingaben.
```

17.6 Lasse Agenten offene Fragen explizit nennen

Ein nützlicher Prompt-Baustein, den ich fast immer dranhänge:

Wenn Informationen fehlen, erfinde sie nicht.
Liste offene Fragen explizit auf.
Unterscheide zwischen:
– blockierende Fragen
– nicht-blockierende Annahmen

17.7 Nutze Reviews als Standard

Keine relevante Änderung ohne Review. In der Regel:

Engineer Agent → Review Agent → Menschliche Freigabe

Bei Architektur:

Product Owner → Architect Agent → Mensch → Engineer → Review

Bei Design:

UX Agent → Mensch → Engineer → Review

17.8 Begrenze Autonomie bewusst

Nicht jeder Agent braucht volle Rechte. Ein bewährter Schnitt:

| Agent | Darf Code ändern? | Darf Architektur ändern? | Darf final entscheiden? |
|---------------|------------------------|--------------------------|-------------------------|
| Product Owner | Nein | Nein | Nein |
| Architect | Nein, außer beauftragt | Vorschlagen | Nein |
| Engineer | Ja | Nein | Im Ticket-Scope |
| Reviewer | Nein | Nein | Review-Empfehlung |
| UX/UI | Nein, außer beauftragt | Nein | Nein |
| Documentation | Ja, Doku | Nein | Nein |

17.9 Dokumentiere Entscheidungen sofort

Wichtige Entscheidungen gehören nicht in Kommentare, sondern in `DECISIONS.md`. Dazu zählen für mich vor allem: Bibliotheksauswahl, Architekturänderungen, Produktumfang, Security-relevante Entscheidungen, Datenmodelländerungen, externe Services und Abweichungen von bisherigen Standards.

17.10 Halte Tasks klein

Agenten arbeiten besser mit kleinen, klar abgegrenzten Aufgaben. Statt „Baue die komplette App“:

1. Erstelle Countdown-Card-Konzept.
2. Erstelle implementierbares Ticket.
3. Implementiere CountdownCard.
4. Implementiere CountdownFormatter.
5. Reviewe UI und Tests.

17.11 Schreibe den Task-Mode immer an den Anfang

Klingt banal, verhindert aber sehr viele Fehler.

Schlecht:

```
Mach mal eine Countdown-Karte.
```

Besser:

```
MODE: PROPOSE
```

```
TASK:
```

```
Entwickle 3 visuelle Richtungen für die Countdown-Karte.
```

```
EINSCHRÄNKUNGEN:
```

- kein Code
- nur Vorschläge
- am Ende auf Auswahl warten

Oder:

```
MODE: EXECUTE
```

```
TASK:
```

```
Implementiere die abgestimmte Countdown-Karte gemäß `docs/design/components/countdown-card.md`.
```

18. Zusätzliche sinnvolle Dateien

18.1 DESIGN.md

Bei UI-Projekten unverzichtbar.

```
# DESIGN.md

## 1. Designprinzipien
## 2. Zielwirkung
## 3. Layoutregeln
## 4. Farben
## 5. Typografie
## 6. Komponenten
## 7. Abstände
## 8. Responsives Verhalten
## 9. Animationen
## 10. Barrierefreiheit
## 11. Beispiele
## 12. Anti-Patterns
```

18.2 Komponenten- und Flow-Spezifikationen

Für ChronoX hat sich eine klare Trennung zwischen Komponenten und Flows bewährt, weil Designentscheidungen oft im Ping-Pong mit dem UX/UI Agent entstehen:

```
docs/design/components/countdown-card.md
docs/design/components/event-list.md
docs/design/components/event-detail.md
docs/design/flows/create-countdown.md
```

Eine gute Komponenten-Spezifikation enthält Zweck, visuelle Richtung, Layout, Zustände, Interaktionen, Nicht-Ziele, Akzeptanzkriterien und spätere Ausbaustufen.

18.3 ARCHITECTURE.md

```
# ARCHITECTURE.md

## 1. Überblick
## 2. Systemkontext
## 3. Module
## 4. Datenmodell
## 5. Schnittstellen
## 6. Dependency-Regeln
## 7. Fehlerbehandlung
## 8. Security
## 9. Testing
## 10. Bekannte technische Schulden
```

18.4 DECISIONS.md

Einfaches, aber sehr nützliches Format:

```
# DECISIONS.md

## 2026-04-30 – PDF-Erzeugung serverseitig

### Entscheidung
PDFs werden serverseitig erzeugt.
```

Kontext

Der Report soll reproduzierbar sein und nicht vom Browser des Nutzers abhängen.

Alternativen

1. Browser Print-to-PDF
2. Serverseitige PDF-Erzeugung
3. Externer PDF-Service

Begründung

Serverseitige Erzeugung ist für V1 kontrollierbarer und später besser automatisierbar.

Konsequenzen

- Zusätzliche Backend-Abhängigkeit
- Tests für PDF-Struktur nötig
- Später Branding möglich

18.5 WORKFLOW.md

Empfohlener Inhalt:

WORKFLOW.md

Human-Agent Communication

Humans communicate with agents by creating tasks, commenting on tasks, approving or rejecting approval requests, and responding to `Human Input Required` sections.

Humans should avoid vague comments such as:

- "looks good?"
- "make it better"
- "do what you think"

Instead, humans should answer with explicit decisions:

- "Choose option B."
- "Proceed with implementation."
- "Do not implement yet. Create a smaller V1 proposal."
- "Blocked: clarify architecture first."
- "Approved with changes: ..."
- "Rejected because: ..."

Agent-Agent Communication

Agents communicate through task assignment, @mentions, handoff comments, review requests, generated work products and updates to project documentation.

Agents must always provide enough context for the receiving agent to continue without guessing.

Blocking Questions

If an agent cannot continue safely, it must:

1. stop implementation,
2. set task status to blocked or waiting for human,
3. write a `Human Input Required` section,
4. provide options and a recommendation.

No Hidden Decisions

Agents must not make hidden product, architecture, security, or UX decisions. Important decisions must be documented in task comments or `DECISIONS.md`.

18.6 HEARTBEAT.md

Heartbeats sind das, was Agenten überhaupt zum Arbeiten bringt. Ohne sie passiert nichts — und mit zu vielen passiert vielleicht zu viel. Ich habe in den ersten Wochen mit Paperclip beides erlebt: einmal saß ein Agent eine ganze Nacht im Leerlauf, weil sein Heartbeat versehentlich deaktiviert war. Beim nächsten Versuch lief er alle zwei Minuten los und war innerhalb eines Tages am Budget-Limit. Beide Male hätte mir eine zentrale, dokumentierte Stelle für die Taktung den jeweiligen Fehler erspart.

Genau dafür ist `HEARTBEAT.md` da: eine Datei im Repo, die festhält, **wie oft welcher Agent läuft, was er dabei tun soll und wann er besser stillsteht**. Heartbeats sind ein zentrales Paperclip-Konzept; die Datei darüber ist die Stelle, an der ihr Setup nicht in der UI versteckt bleibt, sondern reviewbar und versionierbar wird — neben `AGENTS.md` und `WORKFLOW.md`.

Hintergrund: Den Dateinamen habe ich von **OpenClaw** übernommen, das eine `HEARTBEAT.md` als kurze Heartbeat-Checkliste für einzelne Agenten kennt (kleine Erinnerungsliste, die der Agent in seinen Prompt zieht — siehe Quellen, Kapitel 25). Was hier folgt, ist eine **Erweiterung** dieser Idee: kein Prompt-Schnipsel, sondern ein Setup- und Koordinationsdokument für ein Multi-Agent-Projekt. Wenn Du OpenClaw nutzt, kannst Du beide Dateien parallel führen — eine kompakte für den Agent-Kontext, diese ausführlichere für menschliche Leser und Reviews.

Was ich da hineinschreibe

Pflicht ist für mich die **Taktung pro Agent** — also wie oft jeder aufwacht. Mein Engineer läuft typischerweise alle zehn Minuten, der Reviewer alle dreißig, der Documentation Agent zwei- oder dreimal am Tag. „Engineer so oft wie möglich, Reviewer so selten wie möglich“ funktioniert in der Praxis nicht: Reviews liegen dann stundenlang, oder der Reviewer prüft Sachen, die noch im Schreibvorgang sind.

Dazu gehört, **was beim Heartbeat überhaupt passieren soll**. Inbox prüfen, neue Tasks aufnehmen, blockierte Tasks erneut versuchen, hängende Reviews eskalieren — klingt trivial, aber wenn man es nicht aufschreibt, macht jeder Agent etwas leicht Anderes daraus.

Wichtig sind außerdem **Reihenfolge und Abhängigkeiten** zwischen Agenten. Reviewer und Engineer sollten zeitlich versetzt laufen. Documentation läuft bei mir zuletzt — sie soll Fakten festhalten, nicht Zwischenstände dokumentieren, die in fünf Minuten überholt sind.

Dann **Pause-Regeln**: bei mir laufen Agenten nur Mo-Fr von 8 bis 20 Uhr. Wer freitagabends ein Feature freigibt, will montagsmorgens nicht achtzig Folge-Tasks im Inbox finden, von denen die Hälfte am Wochenende nochmal überholt wurde.

Zum Schluss zwei Dinge, die ich erst nach den ersten Schmerzen ergänzt habe: ein **Kostenbezug**, also welche Taktung zu welchem monatlichen Budget passt — und eine **Notbremse** für den Fall, dass ein Agent in einer Schleife landet. Bei mir greift sie nach drei identischen Versuchen am selben Task. Das hat schon mehrmals verhindert, dass über Nacht etliche Dollar in derselben Endlosschleife verbrennen.

Beispiel `HEARTBEAT.md` für ChronoX

```
# HEARTBEAT.md
```

```
## Zweck
```

Diese Datei dokumentiert das Heartbeat-Setup für das ChronoX-Projekt. Sie ergänzt ``AGENTS.md`` (Rollen) und ``WORKFLOW.md`` (Prozess) um die zeitliche Dimension: wann arbeiten welche Agenten, wie oft, und unter welchen Stop-Bedingungen.

```
## Globale Regeln
```

- Heartbeats laufen Mo-Fr, 8:00–20:00 Europe/Berlin.
- Wochenende und Nachtzeit: alle Agenten pausiert (außer manueller Trigger).
- Bei zwei aufeinanderfolgenden Fehlläufen eines Agenten: automatischer Pause-Status, Notification an den Menschen.

```
## Taktung pro Agent
```

```
Agent	Taktung	Aufgaben pro Heartbeat
```

| Product Owner Agent | 30 Min | Inbox, neue Ideen-Tasks, offene Klärungen |
| Architect Agent | 60 Min | Anstehende Architektur-Reviews, ADR-Entwürfe |
| Engineer Agent | 10 Min | In-Progress-Tasks fortsetzen, neue EXECUTE-Tasks aufnehmen |
| Review Agent | 30 Min | Review Requests prüfen (versetzt zu Engineer) |
| QA Agent | 60 Min | Neue Akzeptanzkriterien, Regressionsfälle |
| UX/UI Agent | 30 Min | UX-Iterationen, offene Designvarianten |
| Documentation Agent | 4× täglich (10:00, 13:00, 16:00, 19:00) | DECISIONS.md, README, Changelog |

Was beim Heartbeat passiert

Bei jedem Heartbeat führt jeder Agent in dieser Reihenfolge aus (Status-Werte siehe `WORKFLOW.md` §13):

1. Inbox prüfen – neue Zuweisungen, neue @mentions, neue Kommentare.
2. `Waiting for Human` ignorieren – der Mensch ist am Zug.
3. `Blocked` prüfen – Blocker noch aktiv? Ggf. erneut versuchen oder eskalieren.
4. `In Progress` fortsetzen – eigenen Task weiterbearbeiten.
5. Neue `Ready`-Tasks aufnehmen, sofern Kapazität (Budget) vorhanden.

Reihenfolge / Abhängigkeiten

- Reviewer-Heartbeat ist um 5 Minuten gegen den Engineer versetzt, damit Review Requests beim nächsten Reviewer-Lauf sicher vorhanden sind.
- Documentation Agent läuft erst nach Engineer und Reviewer, damit nichts dokumentiert wird, was noch nicht durchgereicht ist.

Kostenbezug

Monatliche Budgets sind in der Paperclip-Agent-Konfiguration gesetzt. Diese Taktung wurde so gewählt, dass alle Agenten im Schnitt unter 70 % ihres Budgets bleiben – der Rest ist Puffer für Spitzenlast (z. B. Release-Vorbereitung).

Pause-Regeln

Was Paperclip mitbringt:

- Manuell pausierbar pro Agent über die Paperclip-UI.
- Auto-Stop am Cost-Limit (100 %).

Zusätzliche Projekt-Konventionen (Wrapper-Skripte, kein Paperclip-Feature):

- Frühe Warnung bei 90 % des Monatsbudgets mit Notification.
- Globaler Pause-Schalter über eine Datei wie `.paperclip/pause`.
- Auto-Pause bei zwei aufeinanderfolgenden Fehlläufen.

Notbremse: Schleifen-Erkennung

Eigene Konvention, in einem Wrapper umgesetzt: Wenn ein Agent dreimal hintereinander denselben Task ohne Fortschritt anfasst (gleicher Output-Hash), wird der Task auf `Blocked` gesetzt und ein `Human Input Required`-Block erzeugt. So vermeiden wir endlose Iterationen, die nur Budget verbrennen.

Reviewzyklus dieser Datei

Bei jeder Taktungsänderung: Eintrag in `DECISIONS.md` mit Datum und Begründung. Die Datei selbst wird einmal pro Quartal mit den Cost-Reports der Vorquartale abgeglichen.

Worauf ich nach den ersten Wochen besonders geachtet habe

Eine zu hohe Taktung bringt nicht mehr Output, sondern nur höhere Kosten. Tasks brauchen Denkpausen, keine zusätzlichen Trigger. Lieber konservativ starten und erst nachjustieren, wenn man sieht, dass Tasks regelmäßig zu lange liegen bleiben – nicht andersherum.

Reviewer und Engineer laufen bei mir nie synchron. Fünf Minuten Versatz reichen, damit der Reviewer beim nächsten

Aufwachen tatsächlich etwas Fertiges vorfindet und nicht in einen halb fertigen Stand reinschaut.

Documentation seltener als der Rest. Diese Agenten sammeln Stoff; zwei oder drei Läufe am Tag reichen, und das spart spürbar Budget.

Und: Wochenenden aus, sofern nichts dagegen spricht. Das ist weniger eine technische als eine soziale Regel — wer am Wochenende mal nicht draufschauen will, soll nicht montags vor einer überfüllten Inbox stehen.

19. Anti-Patterns

Die folgenden Muster habe ich selbst durchgemacht. Sie kosten alle Zeit, Geld oder Vertrauen in das Setup.

19.1 Der „Mach einfach“-Prompt

Mach einfach mal.

Folge: unklare Ziele, erfundene Anforderungen, kaum prüfbares Ergebnis. Besser: erst analysieren lassen, Optionen anfordern, dann entscheiden.

19.2 Zu viele Agenten zu früh

Jeder zusätzliche Agent erzeugt Koordinationsaufwand. Wenn Verantwortlichkeiten verschwimmen, geben drei Agenten widersprüchliche Antworten zur selben Frage. Lieber klein starten und Rollen erst ergänzen, wenn Du sie wirklich vermisst.

19.3 Agenten ohne Grenzen

Ohne klare Änderungsrechte und Eskalationsregeln entstehen ungewollte Großumbauten, Architekturdrift und Scope Creep. Die wenigen Minuten für eine ordentliche `AGENTS.md` zahlen sich x-fach aus.

19.4 Keine Nicht-Ziele

Wenn niemand sagt, was *nicht* gebaut werden soll, bauen Agenten zu viel. V1 wird unnötig groß, Features werden überdesignt. Nicht-Ziele sind oft wichtiger als Ziele.

19.5 Keine Entscheidungshistorie

Ohne `DECISIONS.md` oder ADRs werden alte Diskussionen Wochen später erneut geführt — manchmal mit dem entgegengesetzten Ergebnis. Der Kontext geht verloren.

19.6 Chat simulieren statt Arbeit strukturieren

Lange Kommentarverläufe ohne klare Status, ohne strukturierte Rückfragen, ohne explizite Entscheidungen — das ist der Versuch, Slack in Paperclip nachzubauen. Funktioniert nicht. Lieber kleine Tasks, klare Status, blockierende Rückfragen, dokumentierte Entscheidungen.

20. Praktische Prompt- und Task-Bibliothek

20.1 Task mit Mode starten

```
MODE: [PROPOSE | ITERATIVE | EXECUTE]

TASK:
[Was soll der Agent tun?]

KONTEXT:
[Warum ist die Aufgabe wichtig? Was ist bereits entschieden?]

ERWARTETE AUSGABE:
[Was soll konkret zurückkommen?]

EINSCHRÄNKUNGEN:
[Was darf nicht getan werden?]

RELEVANTE DATEIEN:
[Welche Dateien soll der Agent beachten?]

ABSCHLUSS:
[Wie soll der Agent stoppen oder berichten?]
```

20.2 Feature klären

```
MODE: PROPOSE

Bitte arbeite als Product Owner Agent.

Ich habe folgende Feature-Idee:

[Idee]

Bitte analysiere:
- Zielgruppe, Nutzerproblem, Nutzen
- mögliche Varianten, Risiken
- offene Fragen
- Akzeptanzkriterien für eine V1

Noch keinen Code ändern.
```

20.3 Architektur prüfen

```
MODE: PROPOSE

Bitte arbeite als Architect Agent.

Prüfe dieses Feature:

[Feature]

Bitte liefere:
- technische Optionen mit Vor- und Nachteilen
- Risiken
- empfohlene V1-Lösung
- Auswirkungen auf bestehende Architektur
- notwendige Änderungen an ARCHITECTURE.md oder DECISIONS.md

Noch keinen Code ändern.
```

20.4 Design explorieren

MODE: PROPOSE

Bitte arbeite als UX/UI Agent.

Ich bin mir beim Design noch unsicher.

Bitte liefere 3 unterschiedliche Richtungen:

- minimalistisch
- produktorientiert
- premium/visuell stark

Für jede Richtung: Layout, Komponenten, Nutzerführung, visuelle Wirkung, Vor- und Nachteile.

Noch keinen Code ändern.

20.5 Ticket erzeugen

MODE: EXECUTE

Bitte erstelle ein implementierbares Ticket.

Input:

[Beschreibung]

Das Ticket soll enthalten:

- Titel, Ziel, Nutzerwert
- fachliche Beschreibung
- Akzeptanzkriterien, Nicht-Ziele
- technische Hinweise, Testfälle
- Definition of Done, offene Fragen

20.6 Implementierung starten

MODE: EXECUTE

Bitte arbeite als Engineer Agent im Umsetzungsmodus.

Setze folgendes Ticket um:

[Ticket]

Regeln:

- Scope nicht erweitern.
- Architektur beachten.
- Tests ergänzen.
- Keine ungefragten großen Refactorings.
- Am Ende Zusammenfassung mit Dateien, Tests und Risiken liefern.

20.7 Review durchführen

MODE: EXECUTE

Bitte arbeite als Review Agent.

Reviewe die Änderungen zu folgendem Ticket:

[Ticket]

Prüfe Funktionalität, Akzeptanzkriterien, Architektur, Codequalität, Tests, Security, Wartbarkeit und unnötige Komplexität.

Strukturiere das Ergebnis als:

- Blocker
- Should Fix
- Nice to Have
- Positive Beobachtungen
- Freigabeempfehlung

20.8 Human Input Required

```
## Human Input Required

### Status
Blocked

### Warum ich blockiert bin
[Kurze Erklärung]

### Entscheidung, die benötigt wird
[Konkrete Frage]

### Optionen
1. [Option A]
2. [Option B]
3. [Option C]

### Empfehlung des Agenten
[Empfehlung mit Begründung]

### Was nach der Entscheidung passiert
[Konkreter nächster Schritt]
```

20.9 Handoff an anderen Agenten

```
## Handoff an [Agent]

### Kontext
[Relevanter Hintergrund]

### Ziel
[Was erreicht werden soll]

### Entscheidungen bereits getroffen
[Liste]

### Offene Fragen
[Liste]

### Relevante Dateien
[Liste]

### Erwartete Ausgabe
[Was der empfangende Agent liefern soll]

### Änderungsrechte
[Ja / Nein / Nur Dokumentation / Nur Vorschlag]

### Eskalation
[Wann der Mensch gefragt werden muss]
```

20.10 Approval Request

```
## Approval Request

### Entscheidung erforderlich
[Konkrete Entscheidung]

### Kontext
[Warum diese Entscheidung nötig ist]

### Optionen

#### Option A
Beschreibung

Vorteile:
- ...

Nachteile:
- ...

#### Option B
Beschreibung

Vorteile:
- ...

Nachteile:
- ...

### Empfehlung
[Agentenempfehlung]

### Benötigte Freigabe
Bitte Option A oder B bestätigen.
Ohne Entscheidung wird nicht weiter umgesetzt.
```

21. Vorschlag für einen späteren LinkedIn-Artikel

Der LinkedIn-Artikel soll nicht das ganze Tutorial wiederholen, sondern neugierig machen.

Möglicher Titel:

Paperclip produktiv nutzen: Warum KI-Agenten Rollen, Grenzen und eine Vision brauchen

Mögliche Struktur:

1. Einstieg mit dem Problem: viele nutzen Agenten wie Chatbots.
2. These: Multi-Agent-Workflows brauchen Organisation, nicht nur Prompts.
3. Paperclip als Steuerungsebene für KI-Agenten.
4. Warum `VISION.md` und `AGENTS.md` entscheidend sind.
5. Warum Kommunikation ohne Chatfenster strukturierter sein muss.
6. Die wichtigsten Agentenrollen.
7. Der beste Workflow: Idee → Optionen → Entscheidung → Umsetzung → Review.
8. Fazit mit Hinweis auf das ausführliche Tutorial.

Hook-Beispiel:

Die meisten KI-Agenten-Projekte scheitern nicht daran, dass die Agenten zu schwach sind.
Sie scheitern daran, dass niemand ihnen sagt, wie sie zusammenarbeiten sollen.

Alternativer Hook:

Paperclip ersetzt nicht das Gespräch mit KI-Agenten.
Es zwingt das Gespräch in eine arbeitsfähige Form:
Aufgaben, Handoffs, Reviews, Entscheidungen und Freigaben.

Möglicher Abschluss:

Meine wichtigste Erkenntnis:
Wer mit Paperclip produktiv arbeiten will, sollte nicht zuerst fragen:
"Welche Agenten kann ich bauen?"
Sondern:
"Welche Organisation brauchen meine Agenten, damit gute Arbeit entsteht?"

22. Checklisten für die praktische Arbeit

Die folgenden Checklisten sind bewusst kompakt gehalten — sie sollen während der Arbeit funktionieren, nicht als zusätzliches Lesematerial.

22.1 Checkliste: Projektstart mit Paperclip

- ☐ Gibt es eine `VISION.md` mit Zielgruppe, Problem, Nutzenversprechen und Nicht-Zielen?
- ☐ Gibt es eine `AGENTS.md` mit Rollen, Rechten, Grenzen und Ausgabeformaten?
- ☐ Gibt es eine `WORKFLOW.md` mit Task-Modes, Handoffs, Reviews und Eskalationsregeln?
- ☐ Gibt es eine `TASKS.md` oder ein vergleichbares Task-Board?
- ☐ Ist klar, welche Agenten Code ändern dürfen?
- ☐ Ist klar, wann der Mensch entscheiden muss?
- ☐ Ist klar, wo Entscheidungen dokumentiert werden?

Wenn ein Agent eine wichtige Entscheidung treffen müsste, die nirgends dokumentiert ist, ist der Task noch nicht reif für `MODE: EXECUTE`.

22.2 Checkliste: Gute Task-Beschreibung

- ☐ `MODE: PROPOSE`, `MODE: ITERATIVE` oder `MODE: EXECUTE`
- ☐ konkretes Ziel
- ☐ relevanten Kontext
- ☐ erwartete Ausgabe
- ☐ Nicht-Ziele oder Einschränkungen
- ☐ relevante Dateien
- ☐ Akzeptanzkriterien, falls Umsetzung erwartet wird
- ☐ klare Abschlussregel

Beispielhafte Abschlussregeln (Sentinel-Tokens, nicht zu verwechseln mit Task-Status-Werten):

```
AWAITING_SELECTION: Bitte wähle A, B oder C.  
ITERATION_1 COMPLETE. Feedback?  
EXECUTE_COMPLETE: Umsetzung abgeschlossen, Tests und Risiken dokumentiert.
```

22.3 Checkliste: Wann `MODE: PROPOSE` ?

- ☐ Die fachliche Richtung ist noch unklar.
- ☐ Mehrere Produktvarianten sind möglich.
- ☐ Ein Design ist noch nicht entschieden.
- ☐ Eine Architekturentscheidung muss vorbereitet werden.
- ☐ Aufwand, Nutzen und Risiko sollen verglichen werden.
- ☐ Du willst bewusst eine Auswahl treffen.

Liegt bereits ein klares, kleines, entscheidungsfreies Ticket vor, ist meist `EXECUTE` besser.

22.4 Checkliste: Wann `MODE: ITERATIVE` ?

- ☐ Eine Richtung ist gewählt, aber Details fehlen.
- ☐ Design, UX oder Text soll gemeinsam verfeinert werden.
- ☐ Du erwartest Zwischenergebnisse und gibst Feedback.
- ☐ Höchstens wenige Iterationen sind nötig.

Pflicht: jede Iteration nummerieren, danach stoppen, nach maximal fünf Iterationen eskalieren oder neu strukturieren.

22.5 Checkliste: Wann **MODE : EXECUTE** ?

- ☐ Ziel und Scope sind klar.
- ☐ Akzeptanzkriterien liegen vor.
- ☐ Relevante Design- oder Architekturentscheidungen sind getroffen.
- ☐ Nicht-Ziele sind dokumentiert.
- ☐ Tests oder Prüfschritte sind benannt.
- ☐ Der Agent weiß, welche Dateien relevant sind.
- ☐ Keine blockierenden Fragen offen.

Müsste der Agent noch Produkt-, UX- oder Architekturentscheidungen erraten, ist **EXECUTE** falsch.

22.6 Checkliste: Handoff zwischen Agenten

- ☐ Kontext
- ☐ Ziel
- ☐ bereits getroffene Entscheidungen
- ☐ offene Fragen
- ☐ relevante Dateien
- ☐ erwartete Ausgabe
- ☐ Änderungsrechte
- ☐ Eskalationsregeln

Ein gutes Handoff ist so vollständig, dass der empfangende Agent nicht den gesamten Kommentarverlauf rekonstruieren muss.

22.7 Checkliste: Review vor Abschluss

- ☐ Alle Akzeptanzkriterien erfüllt?
 - ☐ Scope eingehalten?
 - ☐ Keine ungeplanten Architekturentscheidungen getroffen?
 - ☐ Tests ergänzt oder begründet ausgelassen?
 - ☐ Risiken dokumentiert?
 - ☐ Offene Fragen benannt?
 - ☐ Entscheidung in **DECISIONS.md** nötig?
 - ☐ Spezifikation in **docs/design/...**, **ARCHITECTURE.md** oder **docs/decisions/...** zu aktualisieren?
-

23. Fazit

Effektives Arbeiten mit Paperclip entsteht nicht durch möglichst viele Agenten, sondern durch klare Struktur — eine ordentliche `VISION.md`, eine präzise `AGENTS.md`, definierte Arbeitsmodi, strukturierte Übergaben, Reviews und bewusste Autonomiegrenzen.

Paperclip ist dann produktiv, wenn man es nicht als Chatbot, sondern als Organisationssystem für KI-Arbeit versteht. Du selbst bist nicht mehr der, der jeden Schritt ausführt — eher Board, Product Owner, Entscheider und Qualitätsinstanz.

Die Faustregel, an die ich mich am häufigsten erinnere:

Lasse Agenten nicht raten. Lasse sie Optionen vorbereiten, Entscheidungen dokumentieren und erst danach umsetzen.

Und für die Kommunikation:

Agenten müssen nicht chatten. Sie müssen Arbeit sichtbar, prüfbar und entscheidbar übergeben.

Wenn Du nur eine einzige Sache aus diesem Tutorial mitnimmst, sollte es diese sein: Jeder gute Task beginnt mit einem Modus — `PROPOSE`, `ITERATIVE` oder `EXECUTE`.

24. Anhänge

Die folgenden Dateien bilden ein zusammenhängendes Beispiel-Setup für ChronoX. Sie können direkt im Repo-Root abgelegt oder als Vorlage für ein eigenes Paperclip-Projekt verwendet werden.

Empfohlene Ablage (konsistent mit Kapitel 4 und 11.4):

```
/VISION.md
/AGENTS.md
/WORKFLOW.md
/TASKS.md
/HEARTBEAT.md
/SECURITY.md
docs/
  design/
  decisions/
```

24.1 VISION.md

Beschreibt Produktidee, Zielgruppe, Nicht-Ziele, Kernfunktionen, Qualitätskriterien und Entscheidungsprinzipien für ChronoX. Sie ist die strategische Leitplanke: Wenn ein Agent zwischen Optionen entscheidet, sollte er zuerst prüfen, welche Option besser zur Vision passt.

24.2 AGENTS.md

Beschreibt Rollen, Verantwortlichkeiten, Arbeitsregeln, Task-Modes, Kommunikations- und Eskalationsregeln. Die operative Spielregel-Datei für das Agenten-Team.

24.3 WORKFLOW.md

Beschreibt den konkreten Ablauf von der Idee bis zur Umsetzung — Exploration, Spezifikation, Architekturprüfung, Umsetzung, Review, Dokumentation, menschliche Entscheidungen. Legt außerdem fest, wann `MODE: PROPOSE`, `MODE: ITERATIVE` und `MODE: EXECUTE` zum Einsatz kommen.

24.4 TASKS.md

Ein laufendes Task-Board für ChronoX. Zeigt, wie Aufgaben mit Status, Modus, zuständigem Agenten, Akzeptanzkriterien, Nicht-Zielen und nächsten Schritten dokumentiert werden.

24.5 Verwendung der Anhänge

Mein Vorgehen für ein reales Projekt:

1. `VISION.md` zuerst ausfüllen, damit die strategische Richtung klar ist.
2. `AGENTS.md` an das konkrete Agenten-Setup anpassen.
3. `WORKFLOW.md` als verbindlichen Prozess für Mensch-Agent- und Agent-Agent-Kommunikation verwenden.
4. `TASKS.md` als laufendes Arbeitsboard pflegen.
5. Wichtige Entscheidungen zusätzlich in `docs/decisions/` dokumentieren.

Die Anhänge sind bewusst keine abstrakten Templates, sondern ein vollständiges Beispiel — gedacht zum Kopieren und Anpassen.

25. Quellen

- Tutorial- und Template-Quellen (dieses Repo): <https://github.com/ulrichcech/linkedin-articles/tree/main/paperclip-workflow>
 - Paperclip GitHub Repository: <https://github.com/paperclipai/paperclip>
 - Paperclip Website: <https://papercliping/>
 - Paperclip Documentation: <https://docs.papercliping/> (Mintlify; Inhalte können je nach Build-Stand zeitweise nicht laden)
 - OpenClaw Documentation, Heartbeat-Konzept und `HEARTBEAT.md` : <https://docs.openclaw.ai/gateway/heartbeat>
 - OpenClaw Repository: <https://github.com/openclaw/openclaw>
 - AGENTS.md Open Format: <https://agents.md/>
 - GitHub Issue #49 zu Chat mit Agenten: <https://github.com/paperclipai/paperclip/issues/49> (Feature Request für ein Chat-Interface — dokumentiert, dass Paperclip aktuell kein Chatfenster mitbringt)
-